

Starling

Animated renderings of data structures for teaching

Manual for v0.2.0

Contents

1. Introduction	3
1.1. Quick start	3
2. Architecture	5
2.1. Snapshots, frames, and the renderer	5
2.2. Path identity	6
2.3. Sticky animation accumulation	6
2.4. The cetz integration	7
3. Design decisions	7
3.1. Why frames, not a wrap callback	7
3.2. Three theme layers, one per concern	7
3.3. Frames carry builders, not pre-rendered content	8
3.4. State for ergonomics, per-call for perf	8
3.5. Two-channel captions	9
3.6. The six-frame rotation	9
3.7. Path-based anchors and the cetz tree quirk	10
4. BST animations tour	10
4.1. Static display	11
4.2. Search	11
4.3. Insert	12
4.4. Delete	14
4.5. Rotate	16
4.6. Traversals	18
5. RBT animations tour	22
5.1. Static display	22
5.2. Insert	22
5.3. Delete	25
5.4. Theming the red/black palette	28
6. AVL animations tour	29
6.1. Static display	29
6.2. Insert	30
6.3. Delete	33
6.4. Rotate	37
6.5. Fix-up	37
6.6. Traversals	38
7. Theming	38
7.1. Setting a theme for the whole document	38

7.2.	Per-call overrides	38
7.3.	Performance: state-based theming costs a layout pass	39
8.	Touying composition examples	39
8.1.	Captions in a sidebar	39
8.2.	Driving layout from step metadata	40
9.	Composing with cetz annotations	40
10.	The Op command stream	41
10.1.	Custom shapes and edge anchors	42
11.	API reference	43
11.1.	Render helpers	43
11.2.	Animation kernel	47

1. Introduction

Starling renders animated data structures in Typst, built on top of cetz, typsy, and (optionally) touying. The package is designed for a programming course: it ships a binary search tree today and is structured to grow to heaps, hash tables, and graphs without rewriting the animation kernel.

Every animation in starling is built from the same primitive — an ordered array of Frame records. Each Frame carries a rendered cetz canvas, an optional textual caption, and free-form step metadata. The package supplies a few helpers to collapse the array into a final image, a vertical stack, or an array of subslide figures, but you are free to ignore them and lay frames out however your document needs.

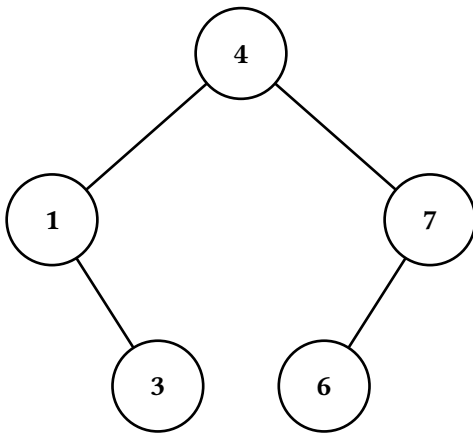
1.1. Quick start

```
#import "@preview/starling:0.2.0" as starling
#import starling: bst

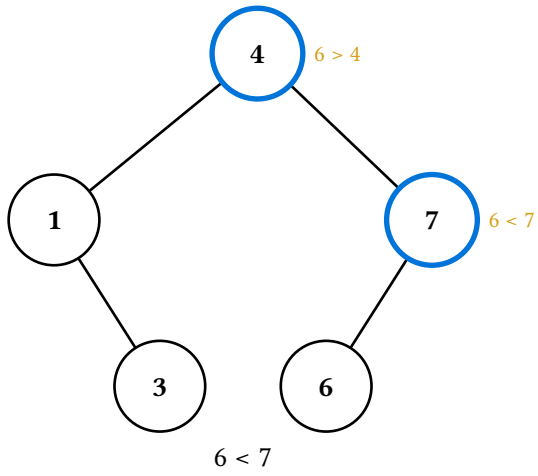
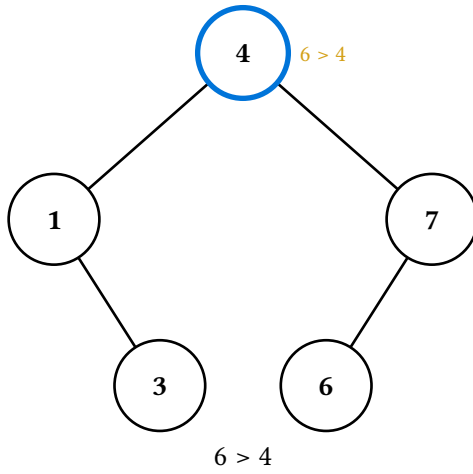
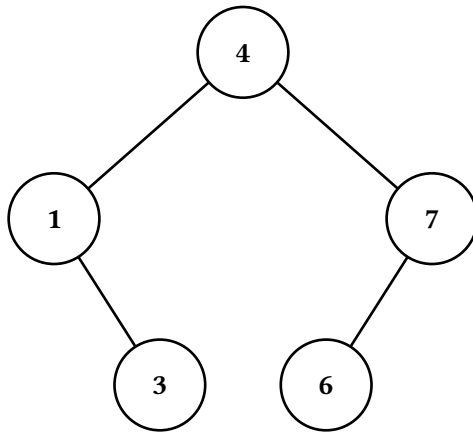
#let t = bst(4, 1, 7, 3, 6)
```

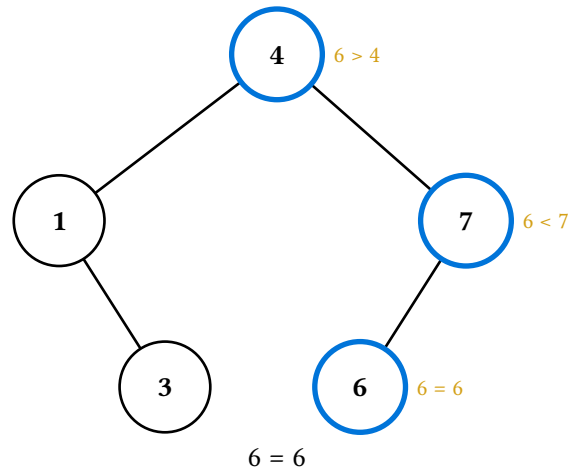
The first argument becomes the root; the rest are inserted in order. For custom node labels, pass a (value, label) 2-tuple in place of a bare value — e.g. `bst((4, [FOUR]), 1, (7, [SEVEN]))`. The lower-level constructor `(BST.new)(value:, label:, left:, right:)` and the `(t.insert-many)(..vals)` method are still available for finer control.

A static render of the tree, taking the final frame from `(t.display)()`:



A full search animation, stacked vertically with per-step captions:





2. Architecture

Starling is layered into three source files, each with a single responsibility:

File	Role
src/tree-anim.typ	The animation kernel — defines Snapshot, Frame, TreeRenderer, draw-tree, Op, and apply-ops. Knows nothing about binary search trees specifically.
src/bst.typ	The BST class — implements pure operations (insert, delete, rotate) and the *-display methods that build animations using the kernel.
src/lib.typ	The public surface — re-exports everything users need, plus the render helpers (last, stacked, figures, canvases-only).

This split lets us add a new data structure (e.g. a heap) by writing just a new file at the bst.typ layer; the kernel does not need to change. Conversely, a power user who wants to assemble custom animations can talk to the kernel directly without touching the BST class at all.

2.1. Snapshots, frames, and the renderer

Four types do most of the work. Their roles are intentionally distinct:

Type	Role
Snapshot	A <i>sparse style overlay</i> for one moment in time — which nodes are filled what colour, which edges are dashed or hidden, which nodes are attached to which nodes. Built up by chaining style-node / style-edge / note-node calls. Pure data; no theme awareness, no content.
TreeRenderer	A tree plus an ordered list of snapshots plus optional caption / step-metadata for each snapshot. The thing you build up while describing an animation.
Frame	The <i>output</i> record: (render, caption, step, alt). render is a builder function (op-theme, render-theme) -> content — not pre-baked content — so callers can resolve theme state at layout time and reuse the same animation across documents with different themes. TreeRenderer.render produces an array of these, one per snapshot. This is what *-display methods return.

Type	Role
Theme dict	One of three layers, in merge order: render-theme (structural defaults like node fill, edge stroke, node colour), op-theme (operation strokes/fills shared across data structures, like search-stroke, success-fill, traversal-palette), and an optional per-DS theme (e.g. rbt-theme's red/black palette). Frames are theme-agnostic until a render helper feeds resolved themes into each frame's render builder.

The data flow, end to end:

```

Snapshot array → TreeRenderer.render
→ Array(Frame) (builders, theme-agnostic)
→ helper resolves theme state once
→ helper calls each (f.render)(op, rt)
→ document content

```

The shape matters: snapshots and frames are both pure data until the final step. That makes them inspectable, slicable, and composable — you can pull a frame out, look at its step metadata, swap its caption, or thread it into a custom layout without rendering. The theming-aware materialisation is concentrated at the helper boundary.

2.2. Path identity

Every node is identified by its position in the tree, encoded as a string of "L" and "R" characters from the root. The root is "", the right child of the root is "R", the left child of "R" is "RL", and so on. Edges are identified by the path of their *child* node — each child has exactly one parent, so this is unambiguous.

Path identity is the linchpin that keeps node and edge styling independent of the tree's value or layout. It also means a node-highlighting animation built for one tree can be reused on a tree of the same shape with different values just by passing a different tree to make-renderer.

The only places that interpret path characters are PathId (the refined string type), _build-cetz-tree (which walks .left and .right), and the BST's own by-value / path-to / resolve methods. Everywhere else treats paths as opaque keys, which leaves room to generalise — n-ary trees would use slash-separated child indices like "0/1/2" or arrays of ints, and only those four places need to change.

2.3. Sticky animation accumulation

make-renderer(tree, sticky: true) is the common case: each new frame pushed via r.push-frame() starts from the previous frame's snapshot. That means a sequence like

```

#let r = starling.make-renderer(t, sticky: true)
#let r = (r.push-with-node)("", stroke: blue + 2pt) // frame 1
#let r = (r.push-with-node)("L", stroke: blue + 2pt) // frame 2
#let r = (r.push-with-node)("LR", stroke: blue + 2pt) // frame 3

```

produces three frames where each highlights one more node than the last — frame 3 has all three highlights, not just LR. This matches the natural mental model for teaching (“we visited these nodes in order”). Set sticky: false if you want each frame to start fresh.

Captions and step metadata do *not* sticky-accumulate; each frame’s caption is whatever was last set on that frame, with no carry-over.

2.4. The `cetz` integration

The animation kernel renders each snapshot via `draw-tree`, which emits `cetz` draw commands using `cetz.tree.tree(...)` for layout. The key choice here is *not* to wrap `draw-tree`’s output in `cetz.canvas` inside the kernel — that wrap happens one layer up, in `_render-canvas`. The two functions are split so a power user can write their own `cetz.canvas` block, call `draw-tree` inside it, and add their own `cetz` annotations alongside the tree (see Section 9).

Anchors for individual nodes are exposed by `path-anchor`, which translates an L/R path string into the `cetz` anchor name produced by `draw-tree`. The fact that this needs a translation function at all is a quirk worth flagging — see Section 3.7.

3. Design decisions

This section explains the major calls that shape the API. None of them are obvious from reading the code, but each was a conscious choice and the reasoning is load-bearing for anyone considering a refactor.

3.1. Why frames, not a wrap callback

The original (v0.1) API let users configure `starling.configure(wrap: alternatives)` to bake touying’s `alternatives` into the `BST` class via closure capture. Each `*-display` method internally called `wrap(..figs)` on the array of figures it generated.

That approach was rigid in two ways. First, the rendering decision was made *inside* the animation method, so the same animation could not appear as a static figure in handouts and as a subslide sequence in slides — you’d configure differently per context. Second, and worse, the animation was always one opaque blob: there was no way to weave the caption “ $6 > 4$ ” alongside other slide content while keeping the visual animation synchronised, because everything lived inside a single `alternatives` mark.

The current API solves both: `*-display` returns the raw frame array, and the helpers (`last`, `stacked`, `figures`) collapse it however the caller wants. Wanting to lay captions in a sidebar while the animation plays? Pull them out yourself:

```
#let frames = (t.search-display)(6)
#grid(columns: 2,
  alternatives(..starling.figures(frames, caption: false)),
  alternatives(..frames.map(f => f.caption)),
)
```

A historical note worth preserving: closure-captured `wrap` was itself chosen over a state-based approach (`std.state`, `typsy`’s `safe-state`) because touying’s `alternatives` is a layout-time “mark” that touying rejects inside context `{ ... }` blocks. With frames as plain data, that constraint no longer applies to the animation API — and state did become viable later for the theming layer (see *State for ergonomics, per-call for perf* below).

3.2. Three theme layers, one per concern

Theming is split into three dicts that each own a different concern:

- **Render theme** (default-render-theme, in tree-anim.typ) holds structural defaults — node fill, node stroke, node text fill, edge stroke, node fill. Anything any data structure would need to render a tree at all.
- **Op theme** (default-op-theme, in op-theme.typ) holds operation- semantic strokes/fills/palettes that apply to *any* data structure with those operations — search-stroke, attention-stroke, success-stroke, settled-stroke, success-fill, danger-stroke, reset-stroke, traversal-palette. BST search and a hypothetical hash-table search share the same search-stroke from this layer.
- **Per-DS theme** (e.g. default-rbt-theme, in rbt.typ) holds styling intrinsic to one data structure — the red/black palette for RBTs is the canonical example. BST itself has no per-DS theme, because BST nodes have no intrinsic differentiation.

The split lets each layer own its own concerns. A new data structure (heap, trie, B-tree) brings its own per-DS theme dict if needed and reuses the render theme and op theme as-is. If operation strokes lived in a BST theme, every new data structure would either duplicate them or grow a coupling to BST.

The render theme also has a clear “lowest in the chain” role: draw-tree falls back to it when neither a snapshot override nor a make-renderer(default-node-style:, default-edge-style:) argument specifies a property. So the merge order, lowest precedence first, is: default-render-theme → user render-theme override → op-theme / per-DS-theme overlays applied per snapshot → renderer defaults → per-snapshot per-path overrides. Each layer adds more specificity.

Strokes throughout the op theme are full stroke dictionaries ((paint:, thickness:, dash:)), not bare colours. That lets users change any aspect of a stroke (dash pattern, cap, width) without us adding more theme keys for each possibility.

3.3. Frames carry builders, not pre-rendered content

Frame.render is a function (op-theme, render-theme) -> content, not a piece of pre-baked content. This is so render helpers (last, stacked, figures) can resolve theme state *once per call* and feed those resolved themes into every frame’s builder, rather than each frame independently resolving theme inside its own context block.

The frame-as-builder shape also has a non-perf benefit: an Array(Frame) is now a portable, theme-agnostic description of an animation. The same frames can render against different themes in different contexts without re-running the *-display method. That’s a cleaner separation than the older (canvas, caption, step) shape, where canvas baked in whatever theme was active at construction time.

A typsy quirk worth noting: typsy auto-injects self into any function-typed field on a class. To keep (frame.render)(op, rt) from getting a spurious third argument, the actual closure is stored as _builder: (fn: ...) — a singleton dict around the function — and exposed through a render method that dereferences it. Users only see (frame.render)(op, rt); the dict wrap is private.

RBT and any future per-DS theme are read inside the frame’s builder itself, not by the lib.typ helpers. The helpers only resolve the two universal layers — op-theme and render-theme — and pass them in; per-DS themes are state-read on demand. That keeps the helper signature stable as more data structures land.

3.4. State for ergonomics, per-call for perf

Theme overrides come in two flavours:

Form	Behaviour
<code>set-op-theme((...)) / set-render-theme((...)) / set-rbt-theme((...))</code>	State-based. One declaration at the top of the document propagates through every subsequent <code>*-display</code> call. Ergonomic and intent-matching, but participates in Typst's state-convergence machinery — see Section 7.3.
<code>(t.search-display)(v, theme: (...))</code>	Per-call. Overrides the theme for one call; ignores state for that call. Verbose if you have many calls, but skips the state-cost path entirely. Run at the no-state baseline.

Both paths exist because there's no single answer. State is right for a document that uses one theme throughout — write `set-op-theme(my-palette)` once, forget about it. Per-call is right when compile speed dominates and the user is happy threading a `let palette = (...) value` through their calls. The library does not pick a winner.

The cost being structural to Typst — not something starling can optimise away — meant choosing whether to expose state at all was a real call. Skipping state would have removed the perf footgun but also removed the ergonomic win. Exposing both, with the cost documented up-front, lets users decide.

3.5. Two-channel captions

For `search-display` and `insert-display`, the comparison string (" $6 > 4$ ") appears in *two* places:

- **Inline**, as a small note drawn beside the highlighted node in the `cetz` canvas. This labels which comparison happened at which node spatially.
- **Caption**, as a textual track on the `Frame` record. This is what `stacked` and `figures` render below the canvas, and what callers pulling captions out for custom layouts read.

The redundancy is deliberate. The inline note is part of the visual: when looking at a single frame, you should be able to see at a glance what the algorithm just did. The caption is a parallel *textual* track for layouts that want narration apart from the animation.

This is why `last(frames)` defaults to `caption: false` — the inline note already labels the node, so adding a caption block below the static figure would be redundant. `stacked` and `figures` default to `caption: true` because animated rendering invites the textual narrative.

3.6. The six-frame rotation

`rotate-display` walks through rotation as six discrete frames: `init`, `pivots`, `break`, `restructure`, `connect`, `settle`. The sequence is calibrated for teaching: each frame answers exactly one question.

step.kind	Question answered
init	What's the starting tree?
pivots	Which two nodes are rotating?
break	Which edges are about to disappear?
restructure	What does the new shape look like, edges aside?
connect	Where do the new edges go?
settle	What does the final, clean tree look like?

An earlier draft had a “dashed red” intermediate between pivots and break (edges going dashed before disappearing). That was dropped because the orange pivot highlight already cues the eye to those edges — the extra frame added length without information.

The restructure frame is the load-bearing one: it shows the new tree shape with the rotated edges still hidden, so the student can see the new positions form *before* the edges connect. Without it, the rotation would collapse into one cut from “broken” to “rotated + highlighted edges”, which is too much information at once.

3.7. Path-based anchors and the cetz tree quirk

`cetz.tree` numbers its nodes internally as 0, 0-0, 0-1, 0-0-1, ... — sibling indices joined by hyphens, with the root being 0. Starling configures `cetz-tree`'s `group-name-prefix` to “node-”, so the resulting anchor names look like `node-0-0-1`. The whole tree also sits inside an outer named group (“tree” by default), so the fully qualified anchor for path “LR” is `tree.node-0-0-1`.

Two things made me reach for path-anchor rather than letting users type that out:

1. The user thinks in “LR”, not “0-0-1”. The translation is mechanical (L → 0, R → 1, prepend the root 0) but it's error-prone to do by hand.
2. The naming scheme is a `cetz-tree` implementation detail. Wrapping it in `path-anchor(path, tree-name:, prefix:)` lets us swap to a different naming scheme later (e.g. if we drop `cetz-tree` for a custom layout) without churn at every annotation call site.

Phantom siblings (used to off-centre lone children — see `_build-cetz-tree`) also get `cetz-tree-internal` anchor names like `node-0-0-0`, but they are hidden at draw time and you generally should not annotate them.

4. BST animations tour

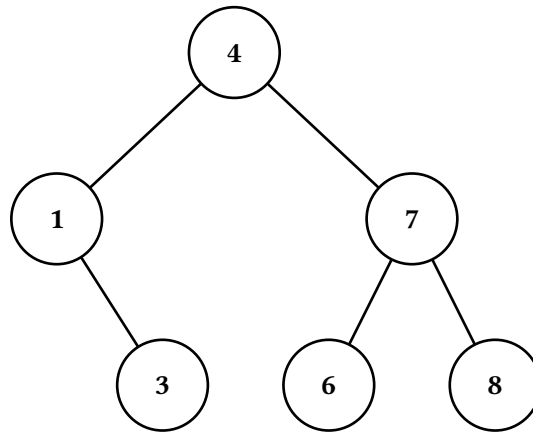
The `BST` class provides five `*-display` methods. Each returns `Array(Frame)`; the examples below pass the result to `starling.stacked` to render every frame vertically with its caption.

We'll use the same sample tree throughout this section, seeded via the `bst(..vals)` factory (first arg = root, rest are inserted):

```
#let t = bst(4, 1, 7, 3, 6, 8)
```

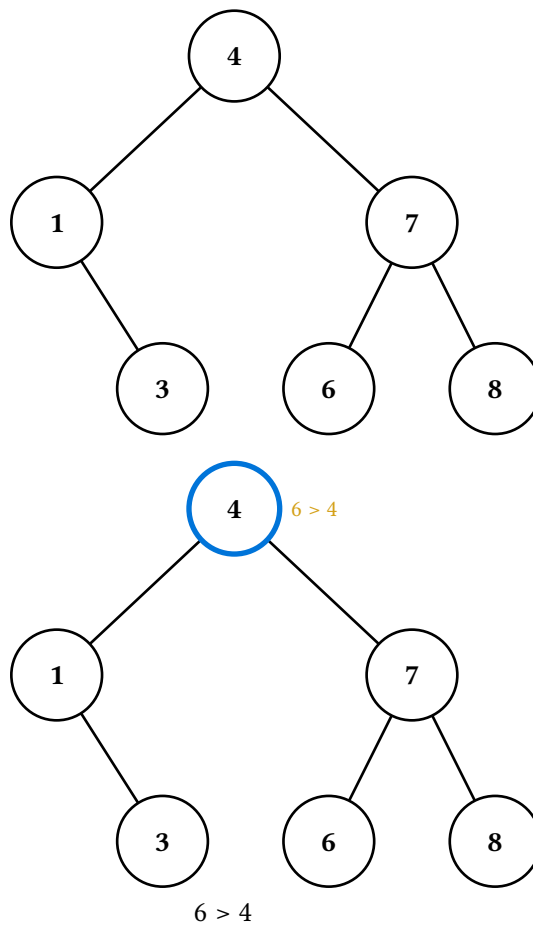
4.1. Static display

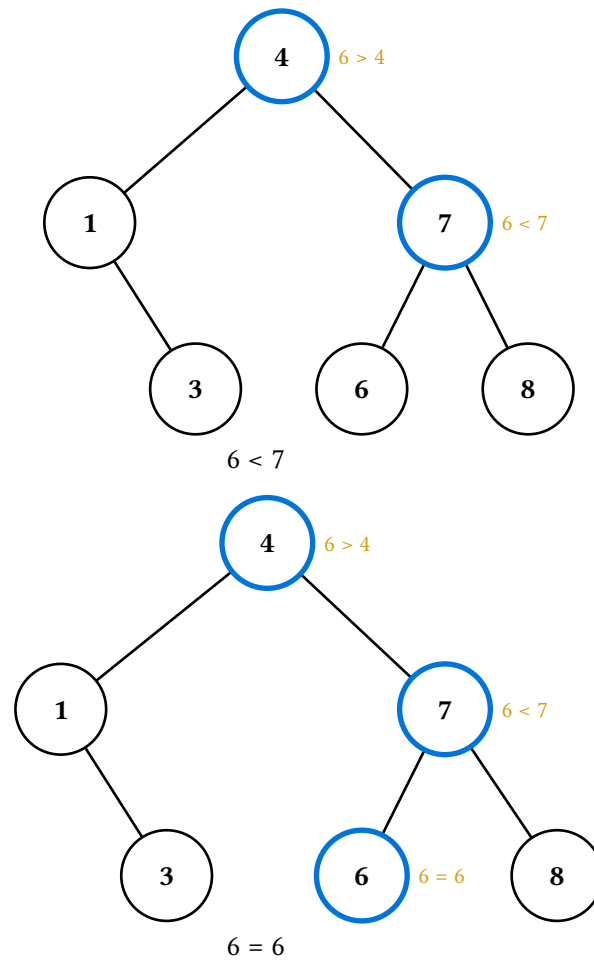
`(t.display)()` returns a one-element frame array — the unmodified tree, with no step metadata.



4.2. Search

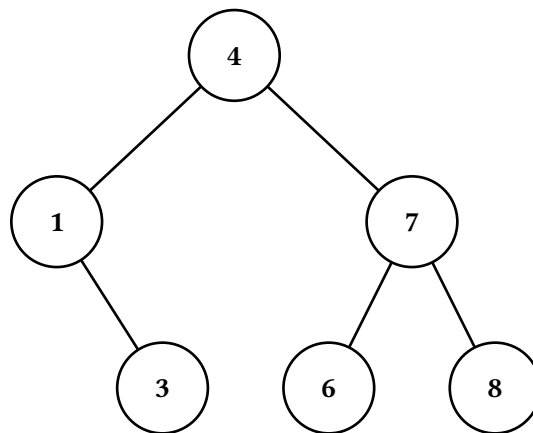
`(t.search-display)(v)` walks the search path, highlighting each node visited and labelling it with the comparison made. `step.kind` is "init" for the initial frame and "compare" for each step; `step.found` records whether the value was reached.

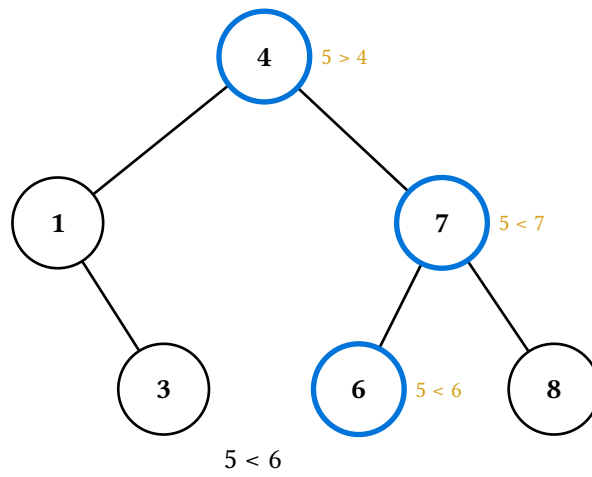
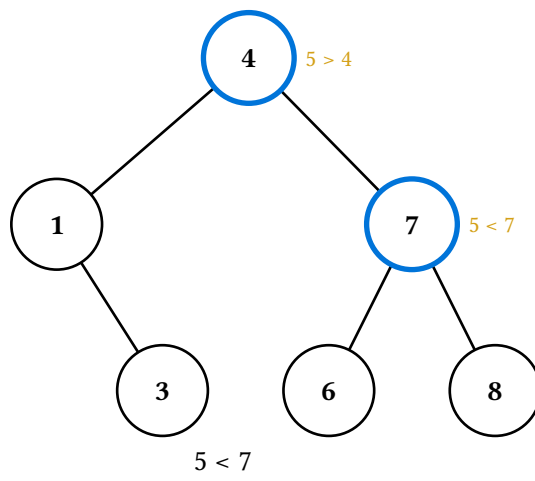
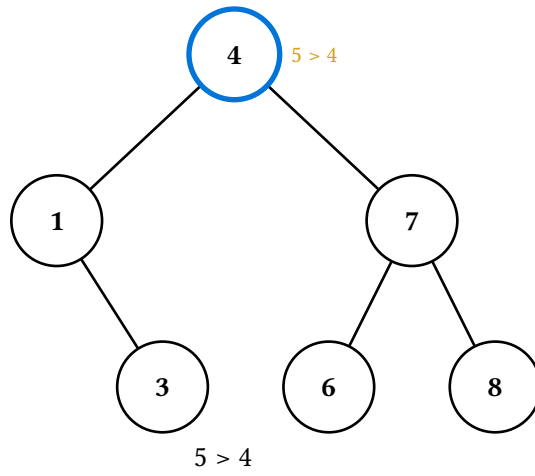


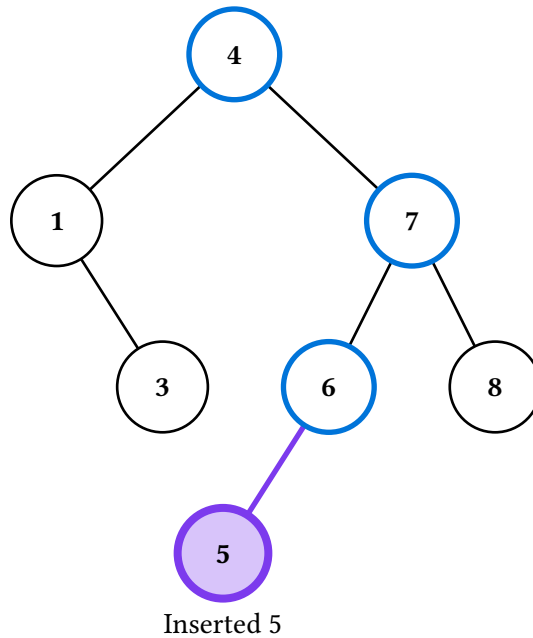


4.3. Insert

`(t.insert-display)(v)` walks the search path as for `search-display`, then transitions to the after-tree with the new node highlighted via `success-stroke / success-fill`. `step.kind` is "init", "compare" (per search step), and finally "inserted".



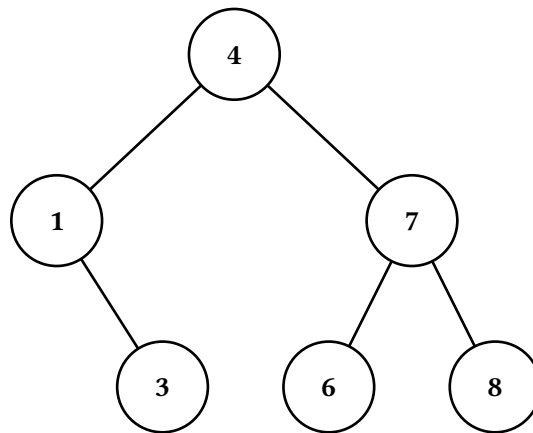


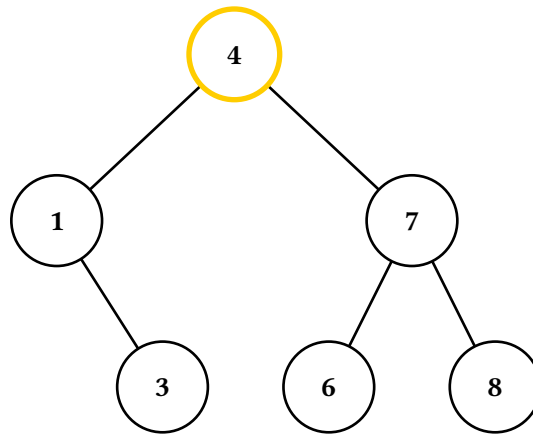


4.4. Delete

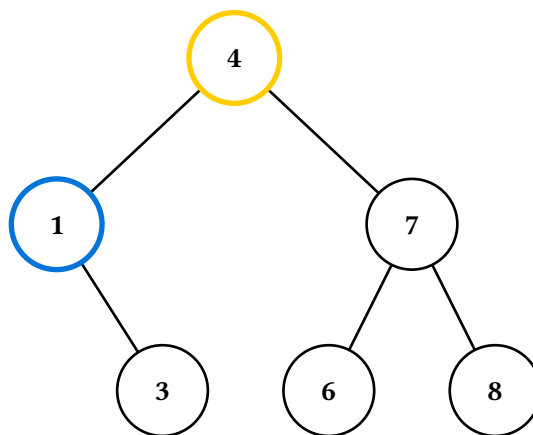
`(t.delete-display)(v)` has three internal cases — leaf, one-child, and two-children — and dispatches on the target's children. `step.kind` ranges over "init", "highlight", "break", "descend", "transfer", and "settle" depending on case.

The two-children deletion is the most complex: it highlights the target, descends into the left subtree to find the in-order predecessor, annotates the value transfer, then jumps to the after-tree with the new root of the affected subtree highlighted via `success-stroke`.

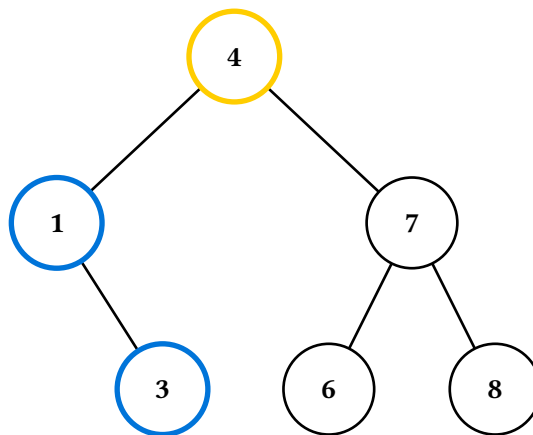




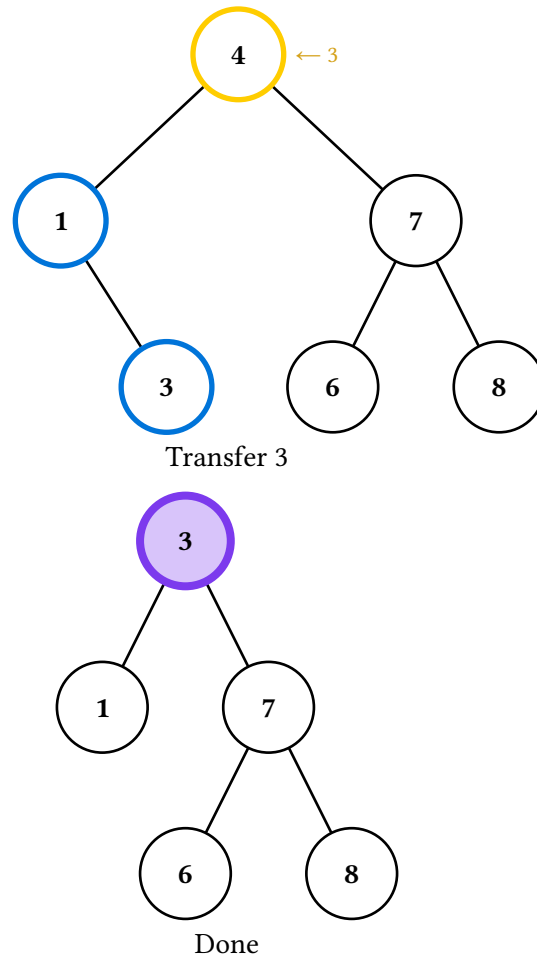
Delete 4



Find predecessor

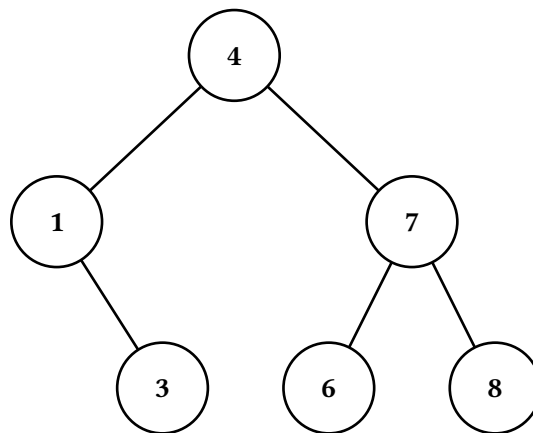


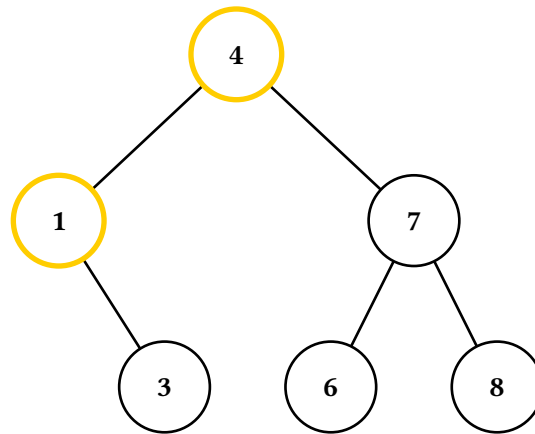
Find predecessor



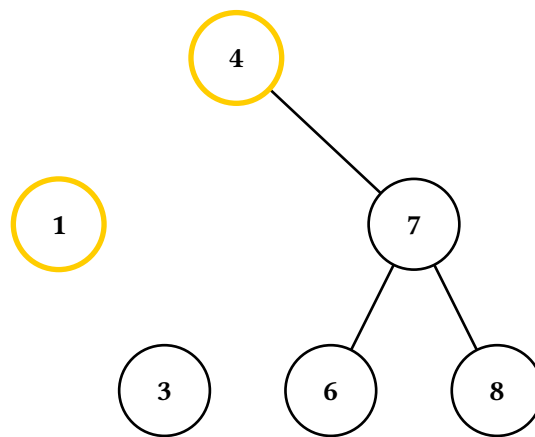
4.5. Rotate

`(t.rotate-display)(c)` rotates around node `c` — any node in the tree with a parent, not just a direct child of the root. The direction (left/right) is inferred from `c`'s position in the BST. The six-frame sequence is described in Section 3.6.

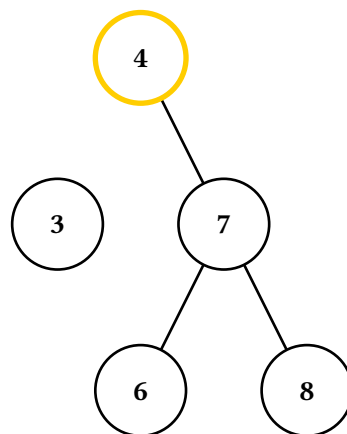




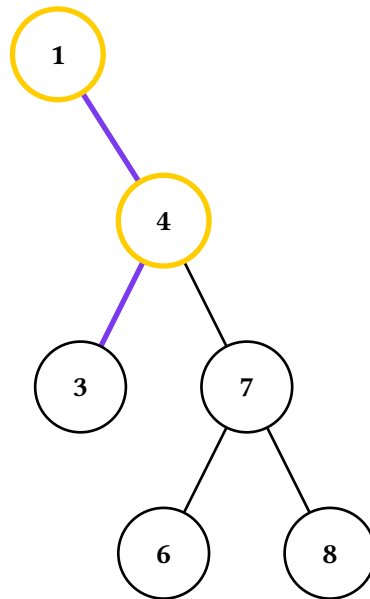
Rotate around 1



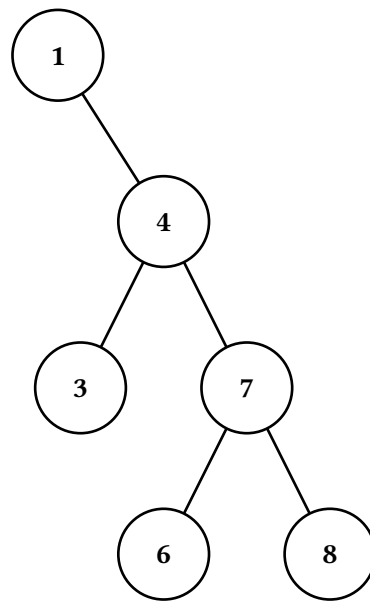
Break edges



Restructure tree



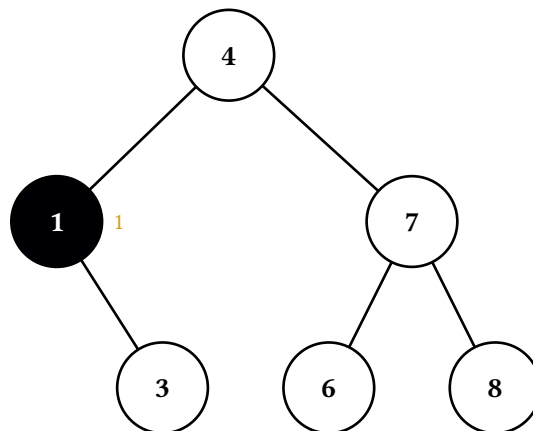
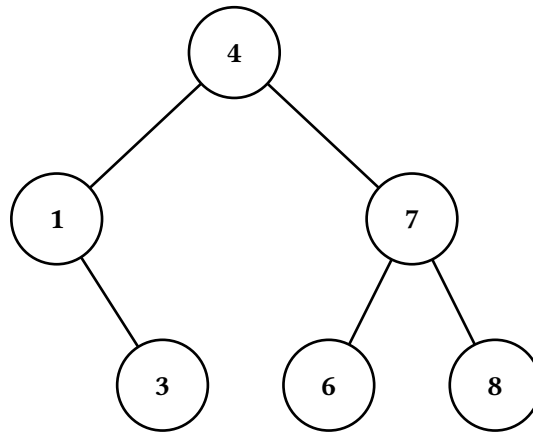
Reconnect edges



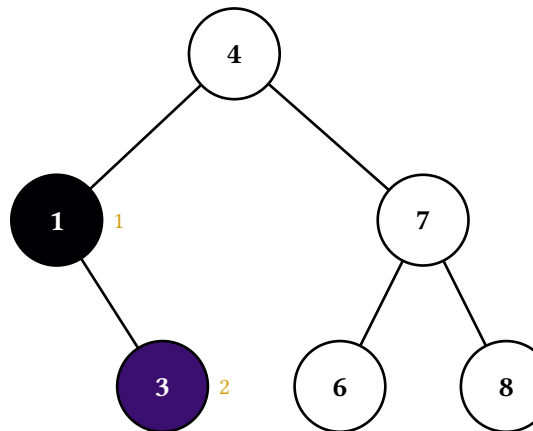
4.6. Traversals

`(t.in-order-display)()`, `(t.pre-order-display)()`, `(t.post-order-display)()`, and `(t.level-order-display)()` animate the four standard traversals. Each returns one frame per visit (plus the initial frame): the visited node is filled with the next color in a perceptually-uniform palette (magma by default) and tagged with a numbered badge marking its visit order, and the caption accumulates the running output sequence. `step.kind` is "init" for the initial frame and "visit" thereafter, with `step.value` and `step.index` (1-indexed) recording each visit. Switch palettes via the theme system (see Theming) — either doc-wide with `set-op-theme((traversal-palette: color.map.viridis))` or per-call with `(t.in-order-display)(theme: (traversal-palette: ...))`.

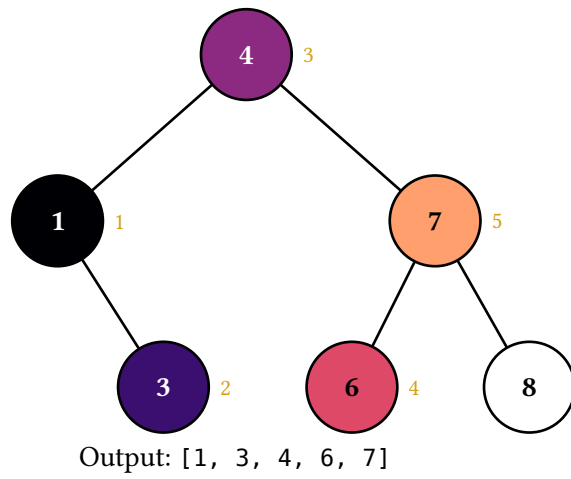
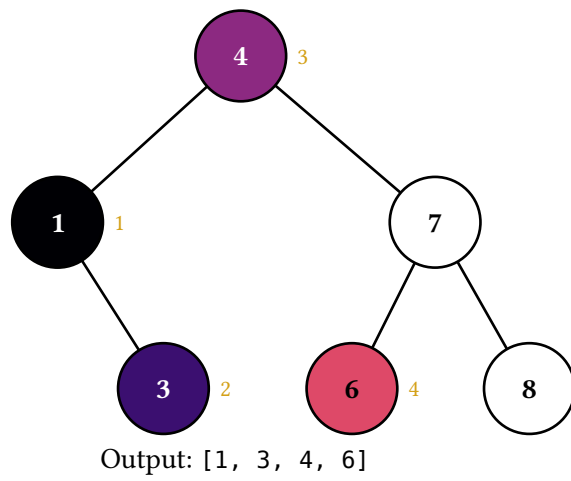
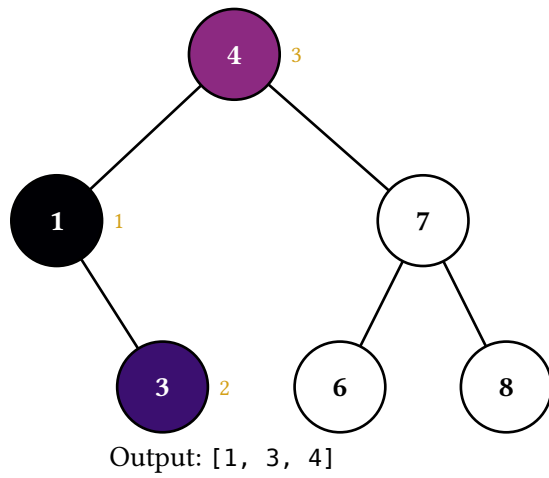
Pure-data variants `(t.in-order)()`, `(t.pre-order)()`, `(t.post-order)()`, and `(t.level-order)()` return the visit sequence as an array of path strings without producing any animation, in case you want to drive a custom layout.

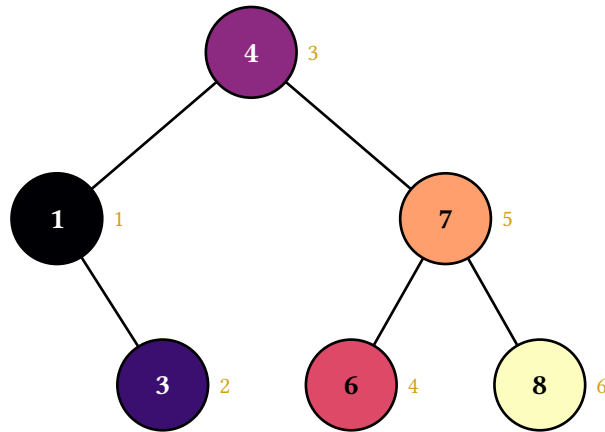


Output: [1]



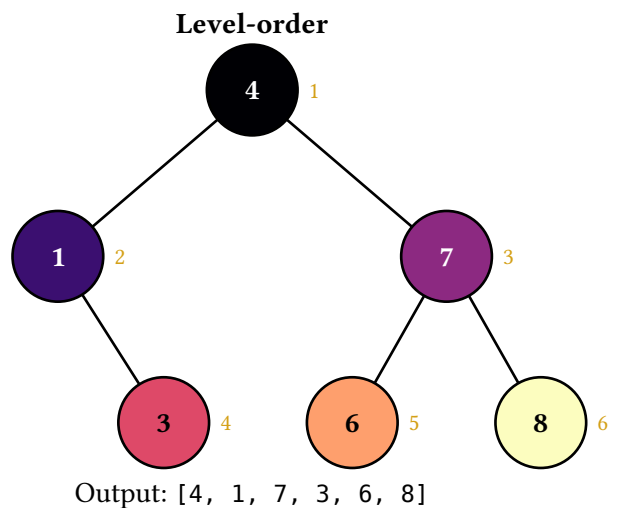
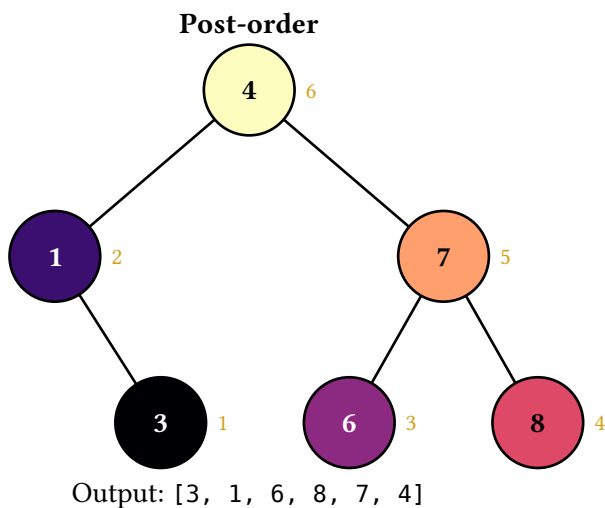
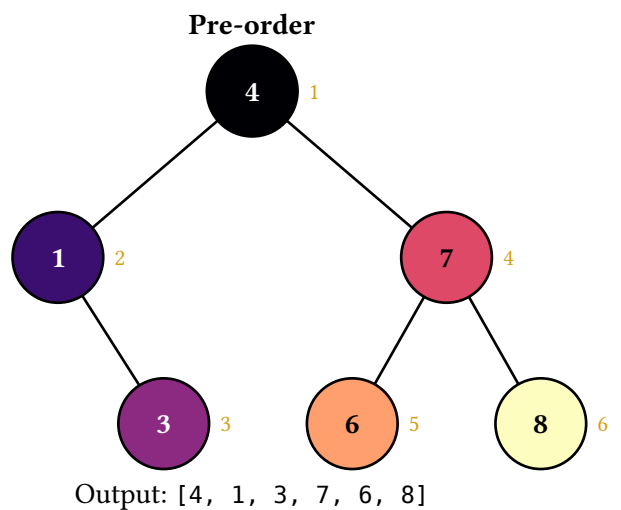
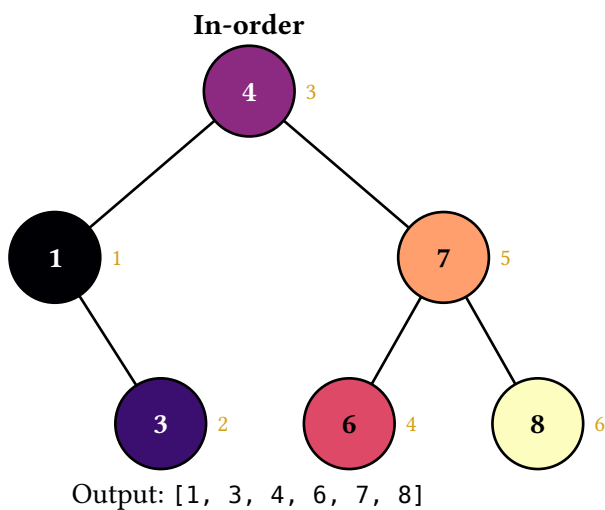
Output: [1, 3]





Output: [1, 3, 4, 6, 7, 8]

The final frame of each traversal carries the algorithm's full color signature. Laying all four out side-by-side gives a fast visual comparison — especially useful as a recall cue in review material where students already understand the algorithms and the gradient pattern reads as “oh right, post-order goes leaves-cool to root-warm”:



5. RBT animations tour

The RBT class ships three `*-display` methods today: `display`, `insert-display`, and `delete-display`. Each returns `Array(Frame)` in the same shape as the BST methods, so the render helpers (`last`, `stacked`, `figures`) and the caption / `step` / `alt` conventions carry over unchanged. The `rbt-theme` palette (red and black fill, stroke, and text-fill) is layered on top of the render theme so every frame's nodes wear their semantic colors automatically; operation-specific highlights from the `op-theme` (`search-stroke` on descent, `attention-stroke` on fix-up pivots, `settled / success strokes` on resolution) compose on top.

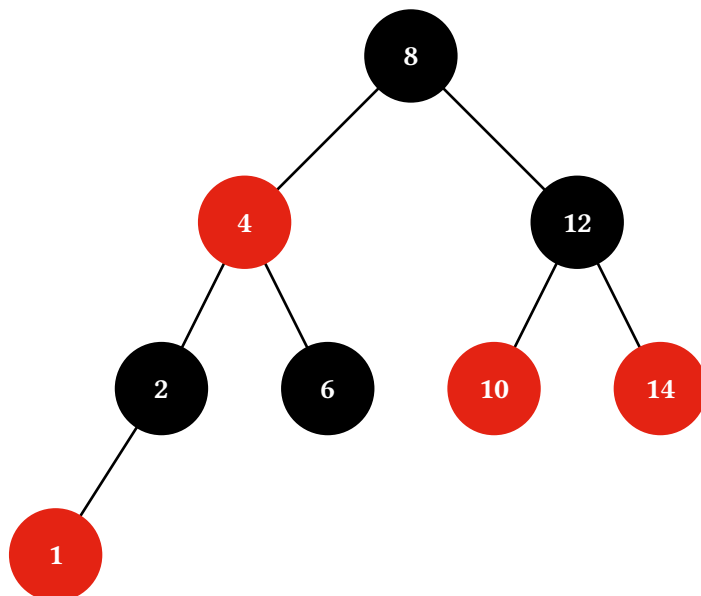
The sample tree throughout this section is seeded via the `rbt(..vals)` factory — first arg becomes the (black) root, the rest are inserted in order so the CLRS fix-ups produce a clean balanced shape with a mix of red and black nodes:

```
#let t = rbt(8, 4, 12, 2, 6, 10, 14, 1)
```

For finer control, the lower-level (`RBT.new`) (`value:`, `label:`, `red:`, `left:`, `right:`) constructor plus (`t.insert-many`) (`..vals`) method are still available — the root takes a `red:` argument because a single-node tree also carries a colour (it must be black after every public operation).

5.1. Static display

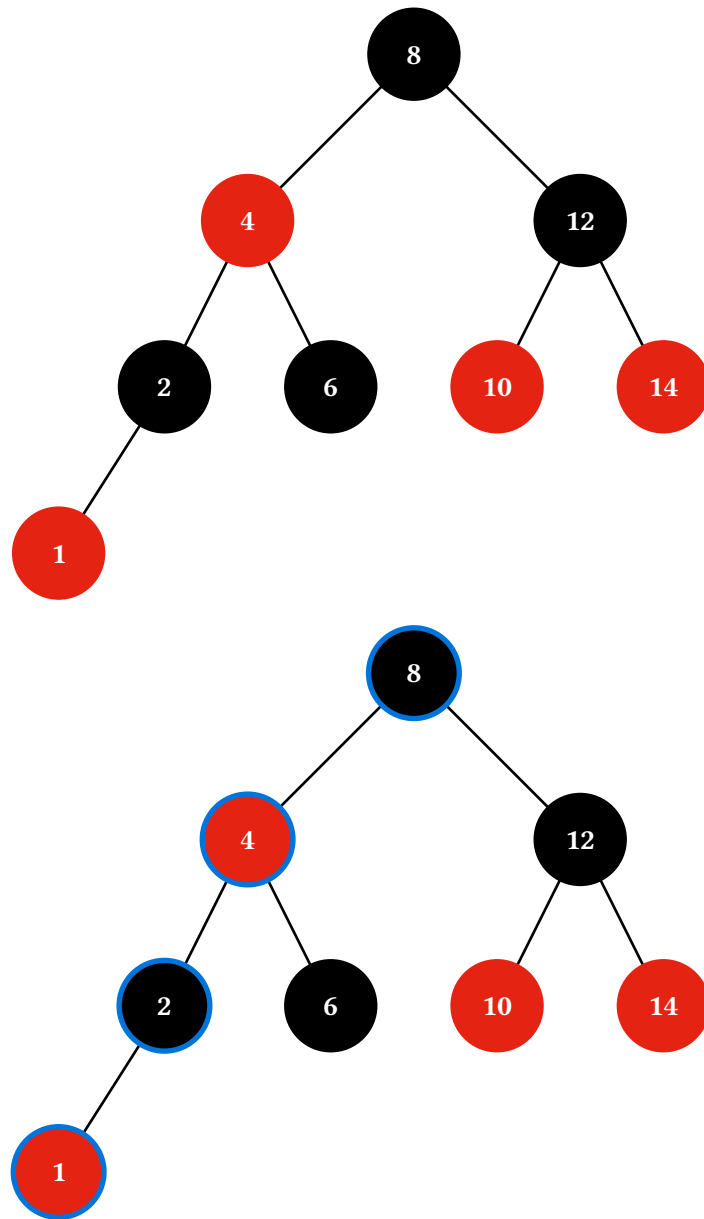
(`t.display`)() returns a one-element frame array — the unmodified tree, with every node coloured by its red field from the active `rbt-theme` palette.



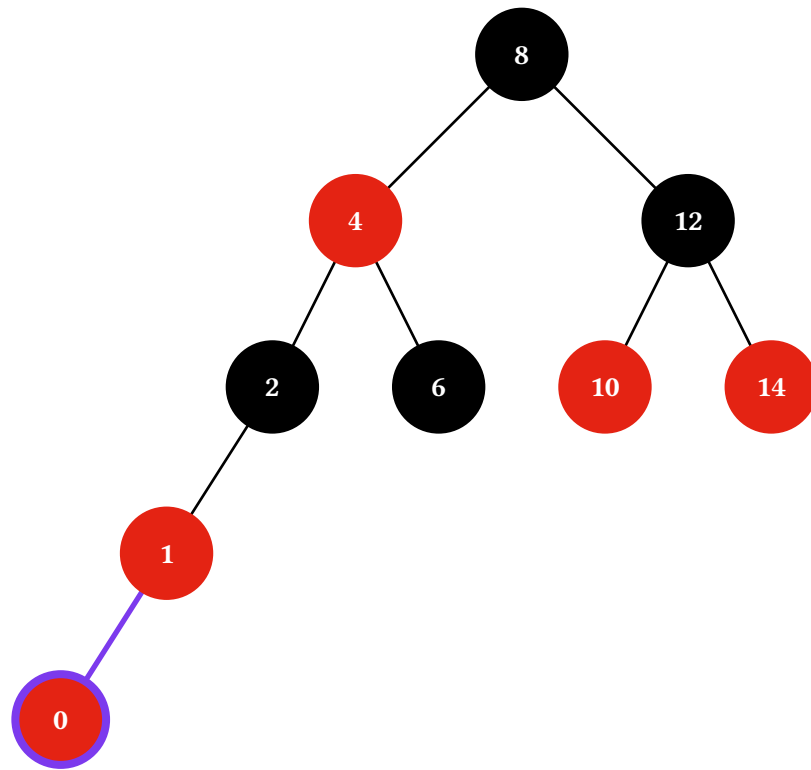
5.2. Insert

(`t.insert-display`)(`v`) traces the CLRS insertion algorithm: a BST descent records each visited node, the new value splices in as a red leaf, then a fix-up loop walks back up the tree applying Case 1 (uncle red — recolour parent and uncle black, grandparent red, continue at the grandparent), Case 2 (zigzag — rotate around the parent to straighten the red-red pair), and Case 3 (straight line — rotate around the grandparent and swap its colour with the new subtree root). The root is blackened at the end if it ended up red. `step.kind` ranges over `"init"`, `"descend"`, `"insert"`, `"check"`, `"recolor"`, `"rotate-zigzag"`, `"rotate-recolor"`, and `"blacken-root"`.

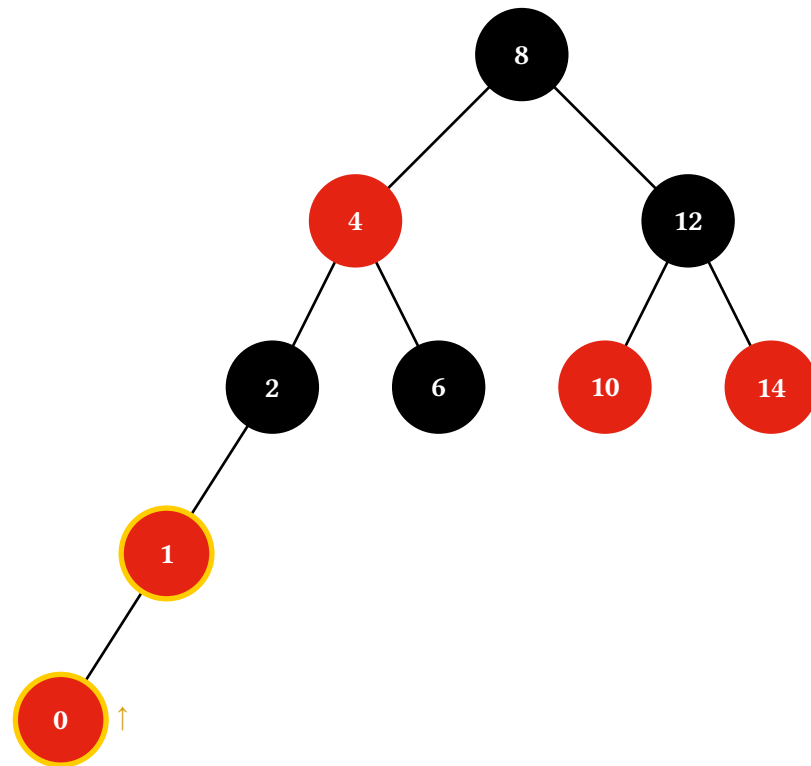
Inserting 0 walks down to the red leaf 1, splices a red 0 beneath it, then resolves the red-red pair via a straight-line Case 3 (rotate around the grandparent 2, swap its colour with the new subtree root 1):



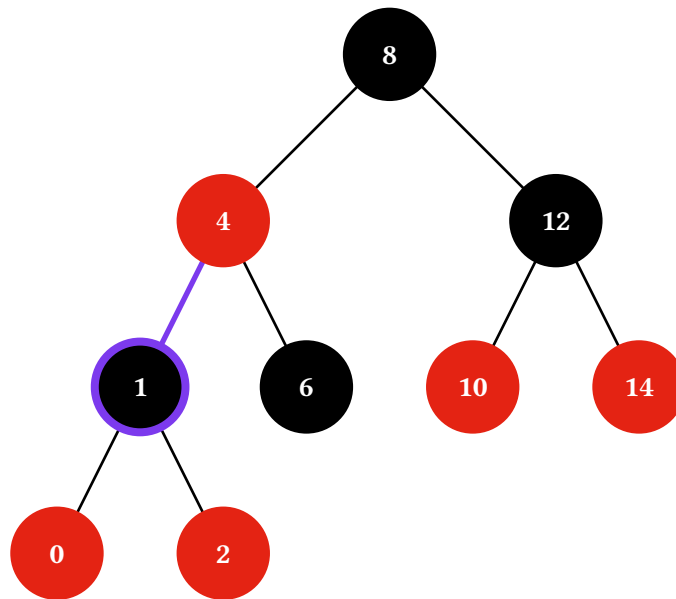
Search for 0



Insert 0 as red leaf



Red-red violation

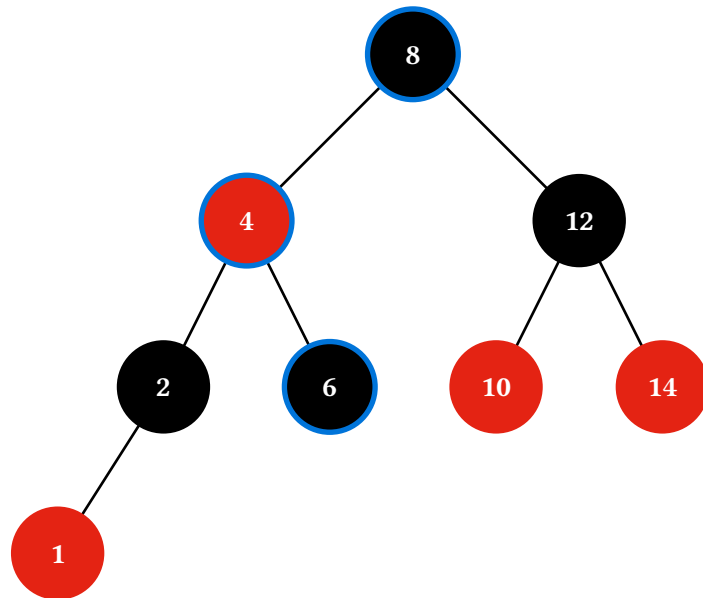
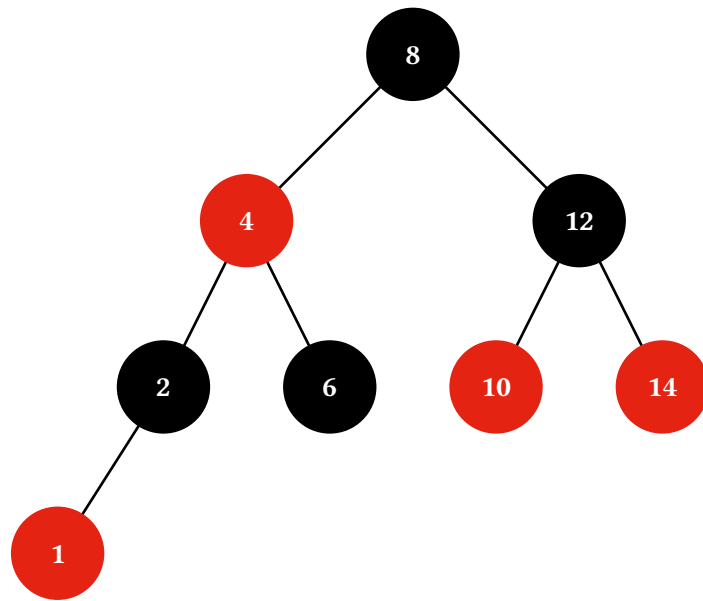


Rotate and color swap

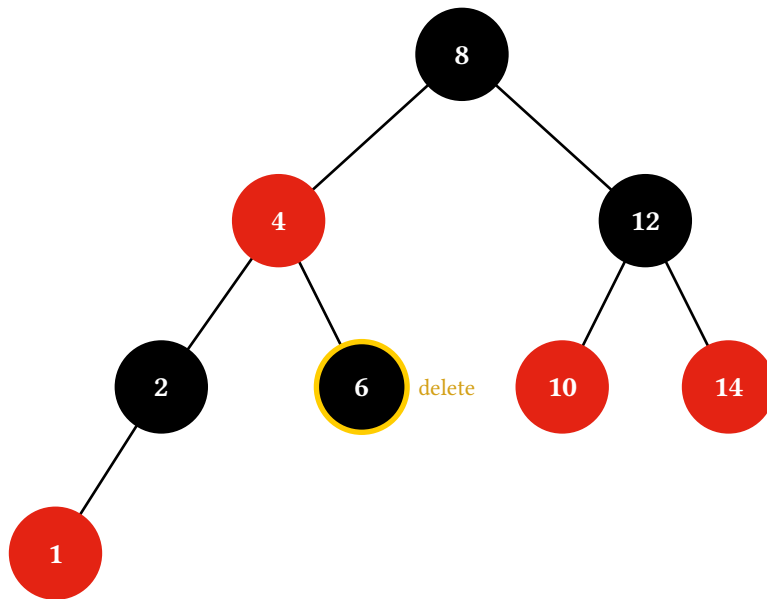
5.3. Delete

`(t.delete-display)(v)` traces a BST search for the target, the in-order predecessor walk if the target has two children, the value transfer, the structural excise, and the four-case rebalancing loop. `step.kind` covers "init", "descend" (a single frame highlighting the full search path; `pass search: true` for one "compare" frame per step instead), "not-found", "mark-target", "find-predecessor", "transfer", "excise", "paint-black-promoted", "paint-black-db", and the fix-up cases. Cases 1, 3, and 4 each emit two frames — a rotation-only intermediate ("case-1-rotate", "case-3-rotate", "case-4-rotate") followed by the recolor ("case-1", "case-3", "case-4") — so the structural pivot lands separately from the color swap. Case 2 is a pure recolor and emits a single "case-2" frame.

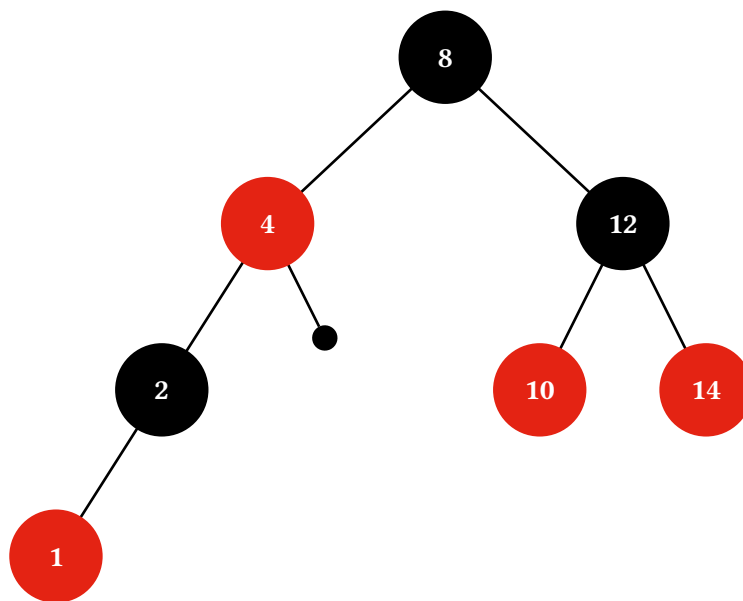
Excising a black node leaves the subtree one black short of the rest of the tree. The animation marks that imbalance with a small filled circle at the child end of the affected edge — the textbook convention. The circle stays in place across each fix-up step that doesn't resolve the missing black; it disappears once a Case 4 rotation drains it or a red ancestor absorbs it. Deleting the black leaf 6 takes the Case 4 branch directly (sibling 2 is black, far nephew 1 is red):



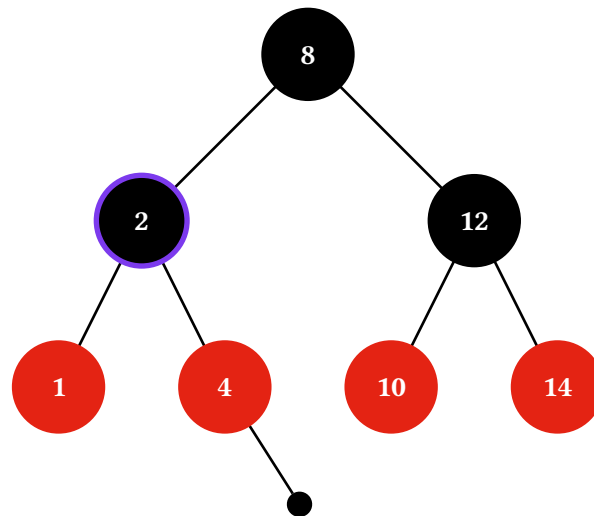
Search for 6



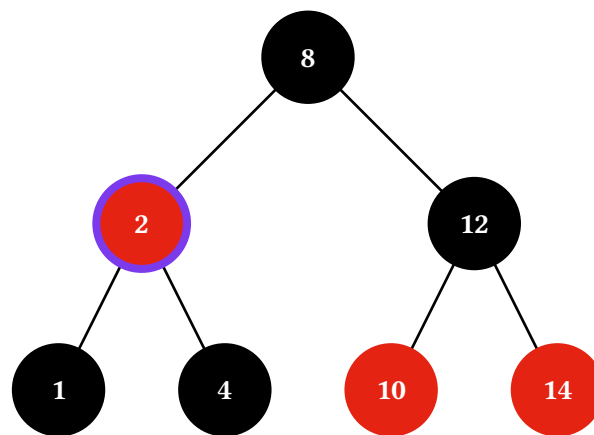
Delete 6



Remove node



Rotate parent



Recolor

5.4. Theming the red/black palette

The `rbt-theme` palette lives in its own state, separate from the render theme and `op` theme, with `default-rbt-theme` as the seed and `set-rbt-theme` / a per-call theme: argument as the two override paths. Recognised keys are `red-fill`, `red-stroke`, `red-text-fill`, `black-fill`, `black-stroke`, and `black-text-fill`; unknown keys panic so typos surface immediately.

Set the palette document-wide via `set-rbt-theme` (state-based, participates in the layout-pass cost — see Section 7.3):

```
#import "@preview/starling:0.2.0": RBT, set-rbt-theme

#set-rbt-theme((
  red-fill: orange,
  red-stroke: orange.darken(20%),
  black-fill: navy,
```

```
    black-stroke: navy,  
  ))
```

Or per-call to skip state and run at the no-state baseline:

```
(t.display)(theme: (red-fill: red.darken(20%)))
```

6. AVL animations tour

The AVL class is a height-balanced BST. Each node carries an extra height field; after every public operation the height is up-to-date and $|h(\text{left}) - h(\text{right})|$ (the **balance factor**) is at most 1. Imbalances are repaired with single or double rotations, dispatched on four cases (LL, RR, LR, RL) — the same ones every textbook walks through.

AVL ships the standard suite of `*-display` methods (`display`, `search-display`, `insert-display`, `delete-display`, `rotate-display`, `fixup-display`, and the four `*-order-display` traversals), all returning `Array(Frame)` so the render helpers and `caption / step / alt` conventions carry across unchanged. AVL has no per-DS theme — animation strokes come from the `op-theme`, structural defaults from the `render theme`.

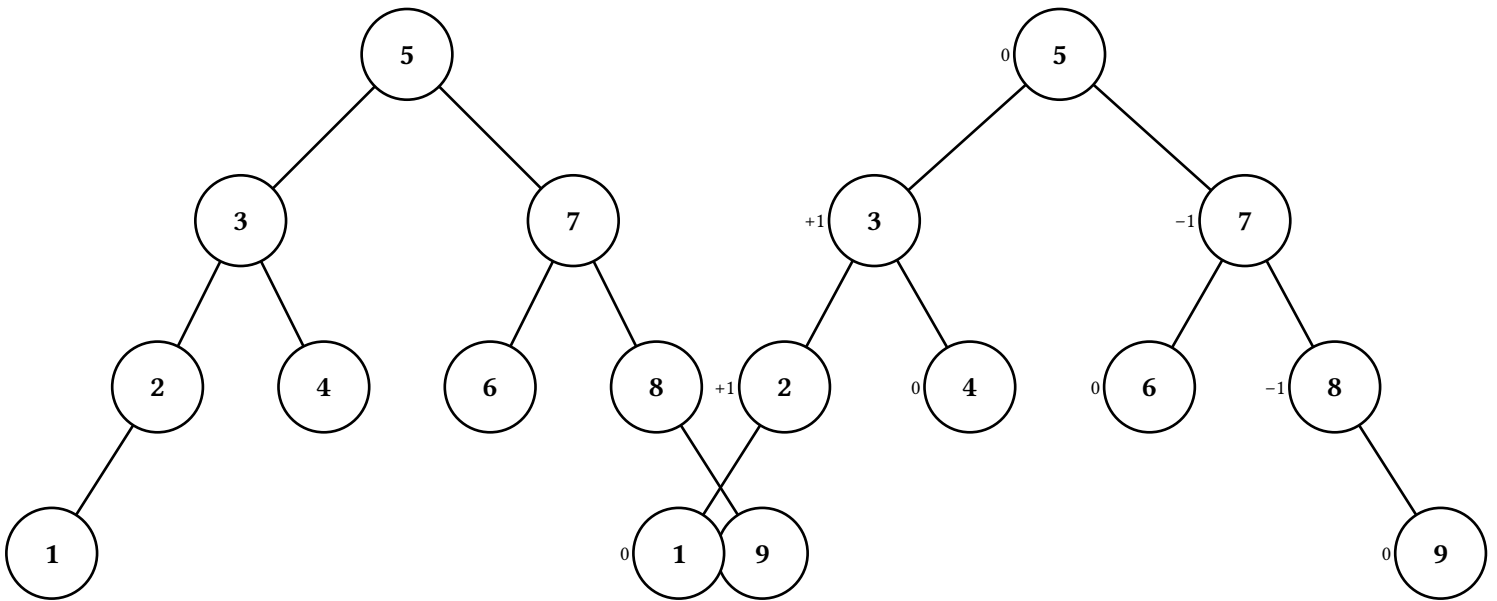
The sample tree throughout this section is seeded via the `avl(..vals)` factory — first arg becomes the root, the rest are inserted in order so the AVL rebalancing produces a clean balanced shape:

```
#let t = avl(5, 3, 7, 2, 4, 6, 8, 1, 9)
```

For finer control, the lower-level `(AVL.new)(value:, label:, height:, left:, right:)` constructor plus `(t.insert-many)(..vals)` method are still available — the root takes a `height:` argument because a single-node tree already has height 1.

6.1. Static display

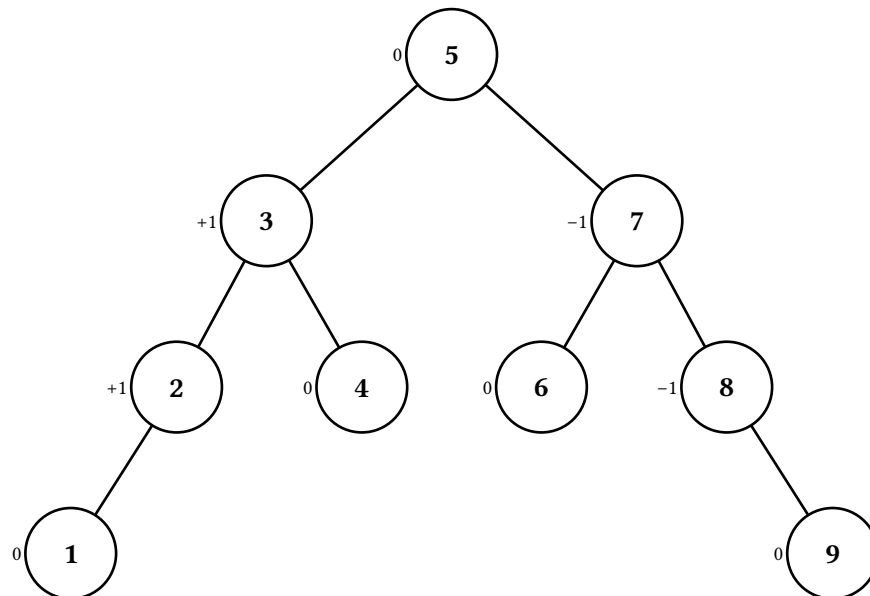
`(t.display)()` returns a one-element frame array — the unmodified tree. Pass `factors: true` to tag every node with its signed balance factor (e.g. `" +1"`, `" 0"`, `" -1"`), drawn just west of the node so it doesn't compete with the label or the operation note slot. The tag layer is reused by every `*-display` method as a base layer (off by default; opt in per call).

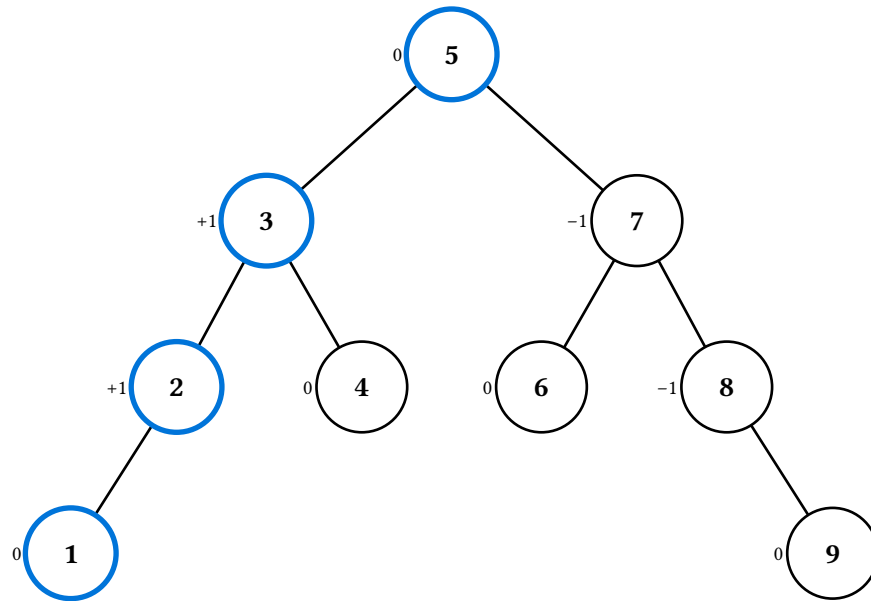


6.2. Insert

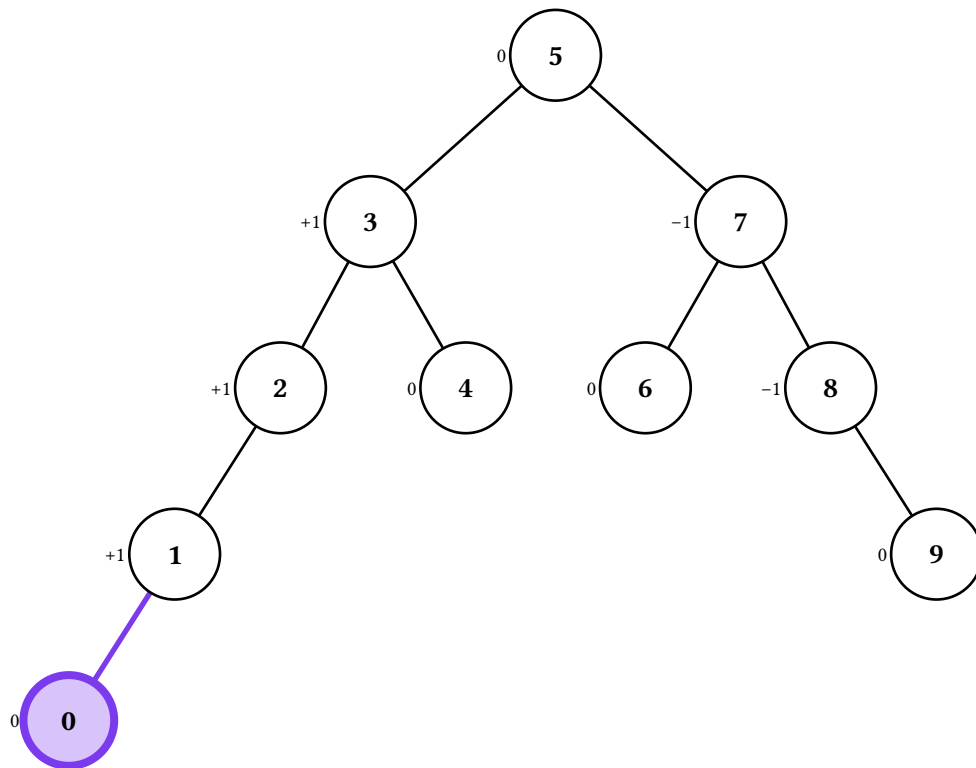
`(t.insert-display)(v)` traces the AVL insertion algorithm. A BST descent records each visited node, the new leaf appears with height 1, then the climb back to the root recomputes each ancestor's height in turn. On the first ancestor whose balance factor reaches ± 2 , the animation labels the imbalance case (LL/LR/RR/RL), applies a child rotation if the case is a zigzag (LR or RL), and finishes with the single rotation at the imbalanced node. After a single insertion the subtree's height returns to its pre-insert value, so the climb stops there. `step.kind` ranges over "init", "descend", "insert", "recompute", "check", "rotate-zigzag", and "rotate-finish".

Inserting 0 into the sample tree triggers an LL fix-up at node 2 (after the climb sees node 1's height grow). The fix-up is a single right rotation:

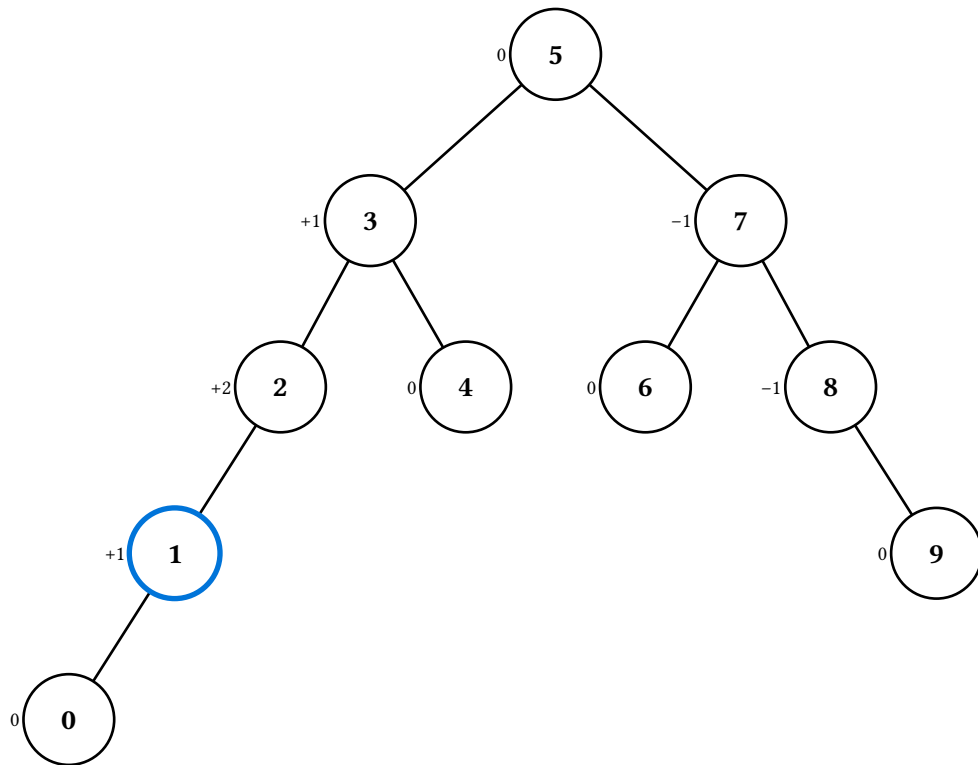




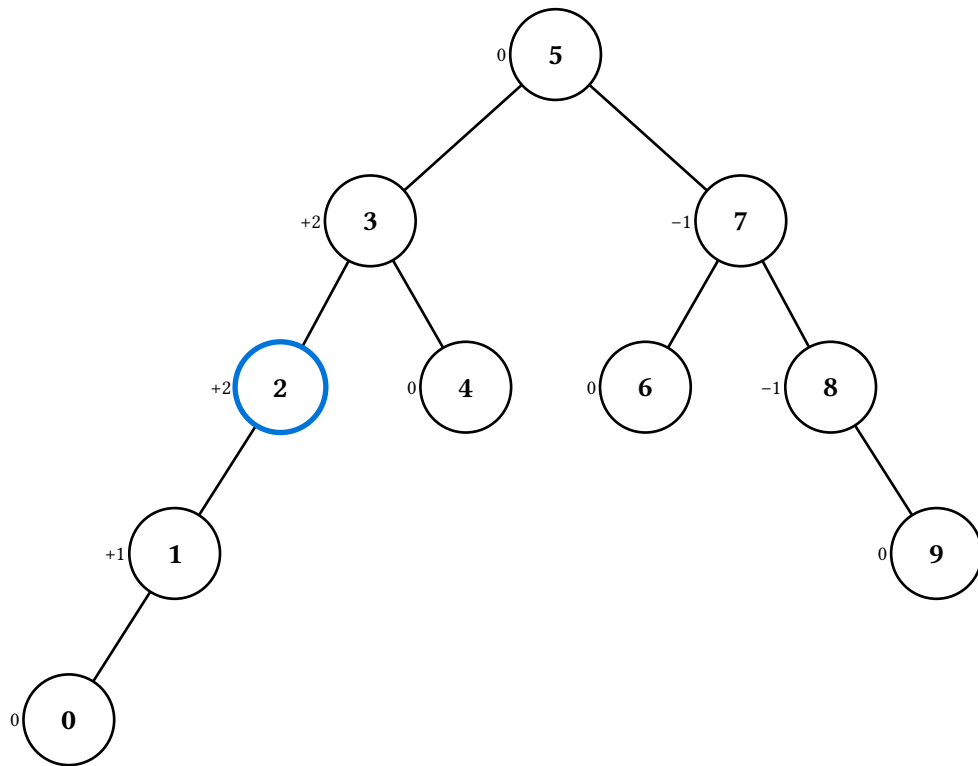
Search for 0



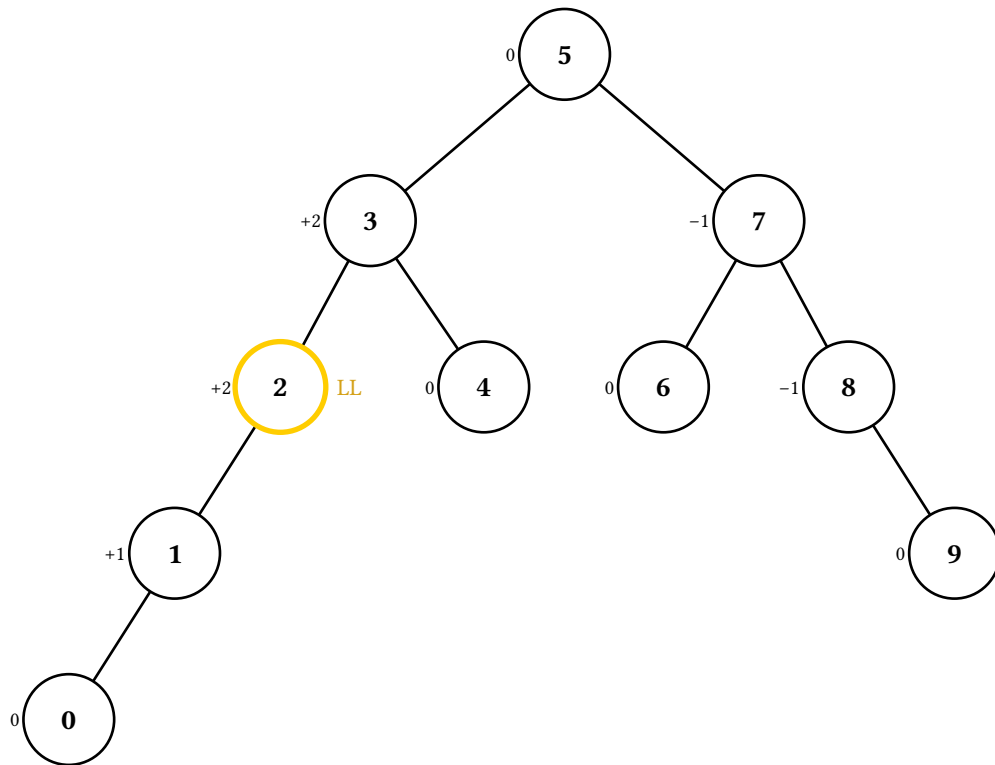
Insert 0 as new leaf



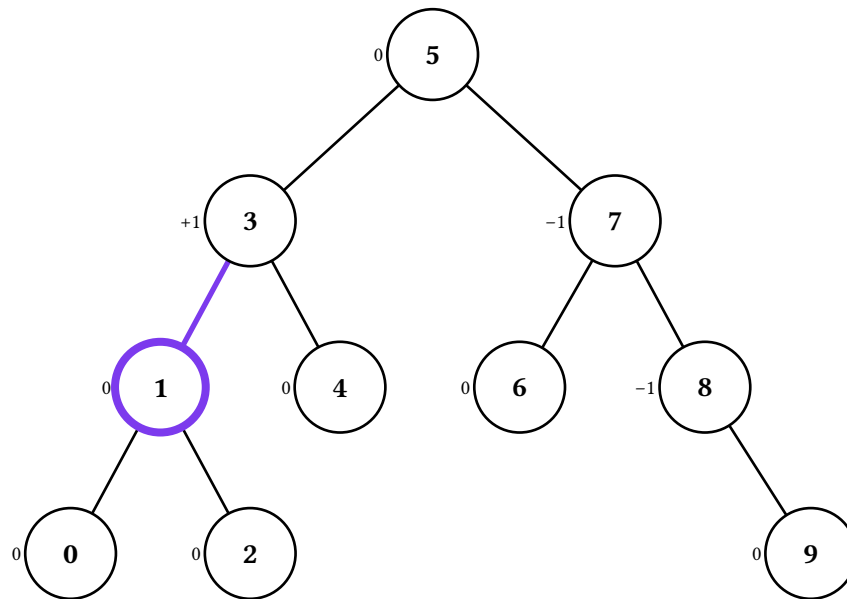
Recompute height = 2



Recompute height = 3



Imbalance: case LL



Rotate (case LL)

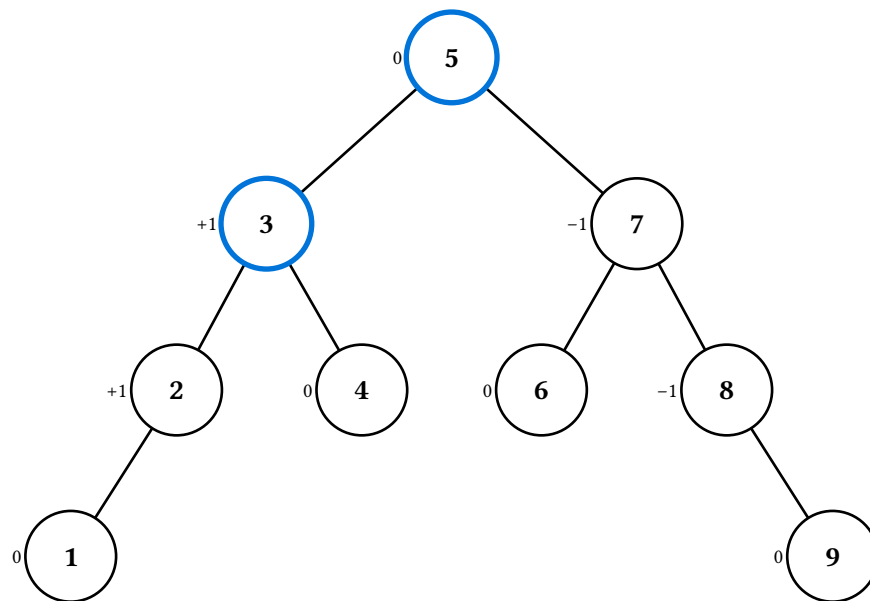
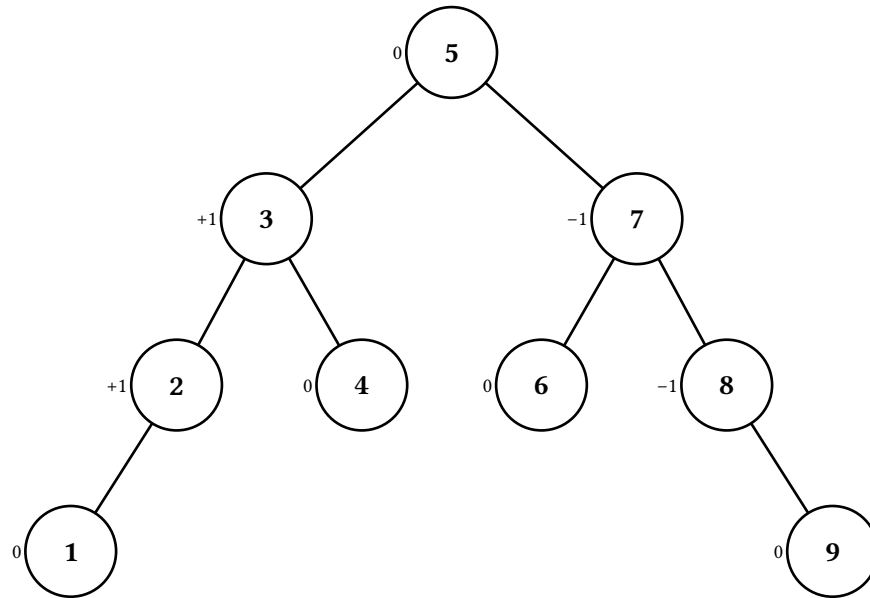
The zigzag cases (LR/RL) emit an extra rotate-zigzag frame between check and rotate-finish for the child rotation that straightens the configuration before the final rotation at the imbalanced node.

6.3. Delete

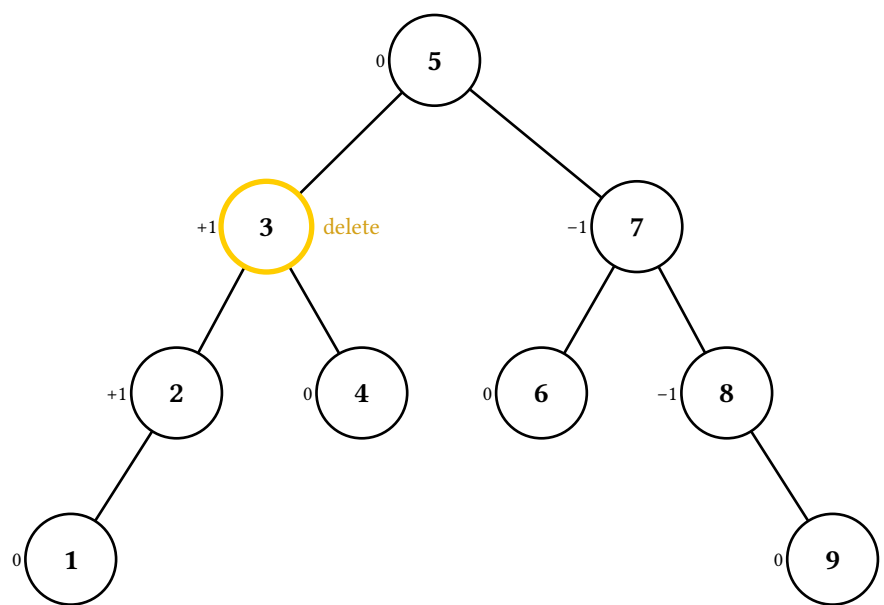
`(t.delete-display) (v)` traces a BST search for the target, the in-order predecessor walk if the target has two children, the value transfer, the structural excise, and then a climb that recomputes heights and rotates at every imbalanced ancestor. Unlike insert, a single deletion can trigger several rotations on the way up

— the loop runs to the root. `step.kind` covers "init", "descend" (single frame; pass search: true for one "compare" per step instead), "not-found", "mark-target", "find-predecessor", "transfer", "excise", and the same "recompute" / "check" / "rotate-zigzag" / "rotate-finish" events insert emits.

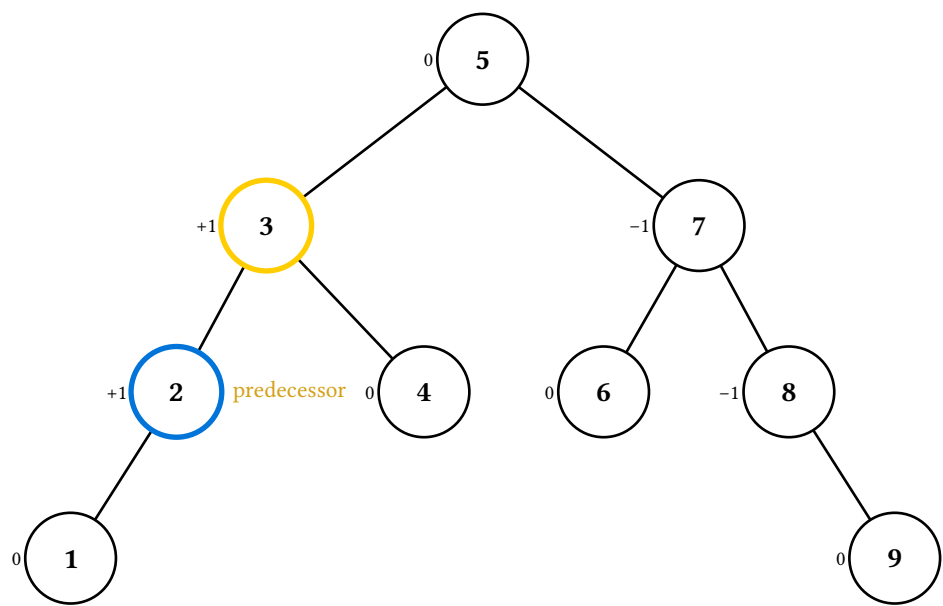
Deleting 3 from the sample tree takes the two-child branch (predecessor transfer from 2) and stays balanced — the climb recomputes heights without triggering any rotation:



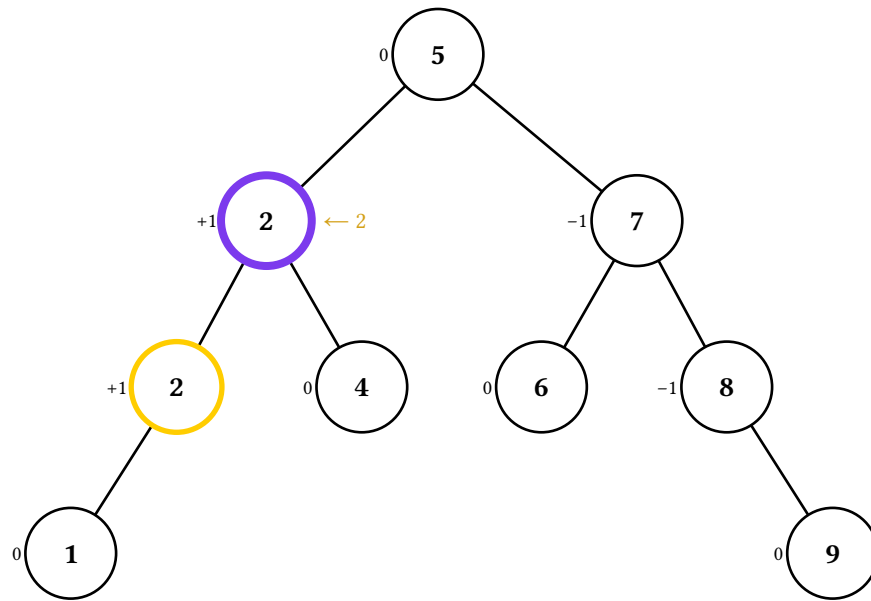
Search for 3



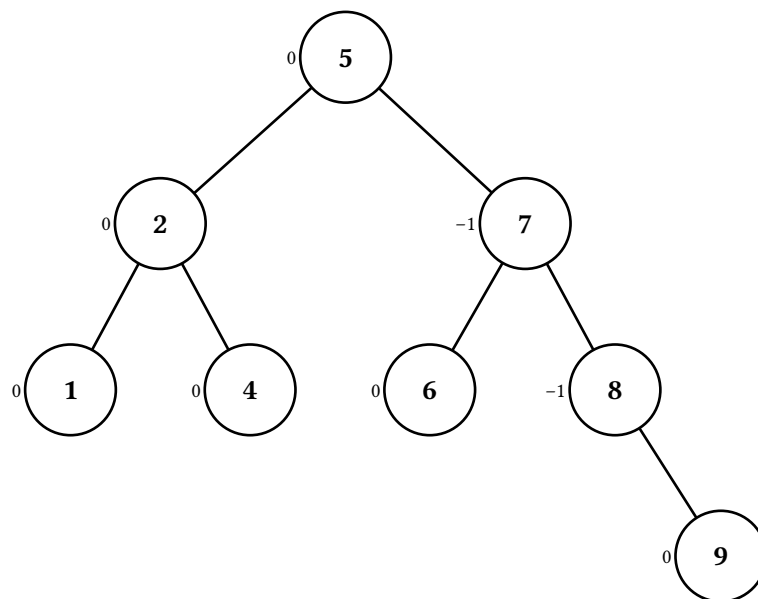
Delete 3



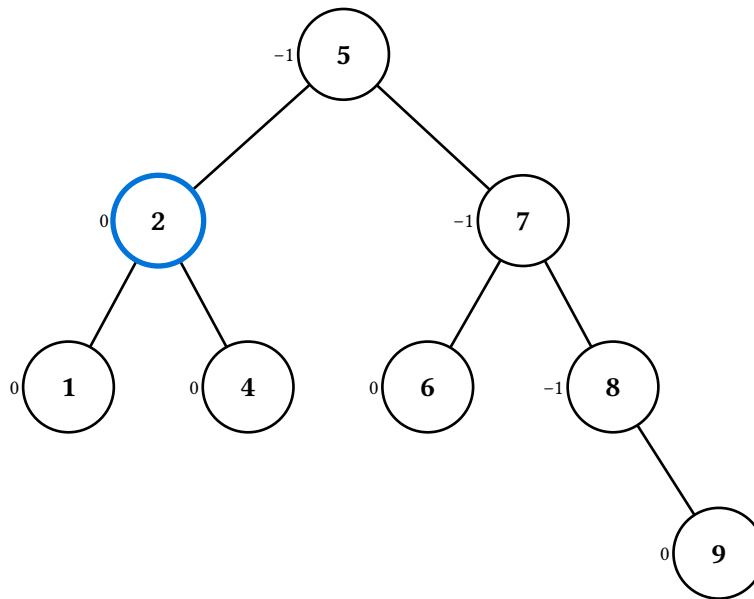
Find predecessor



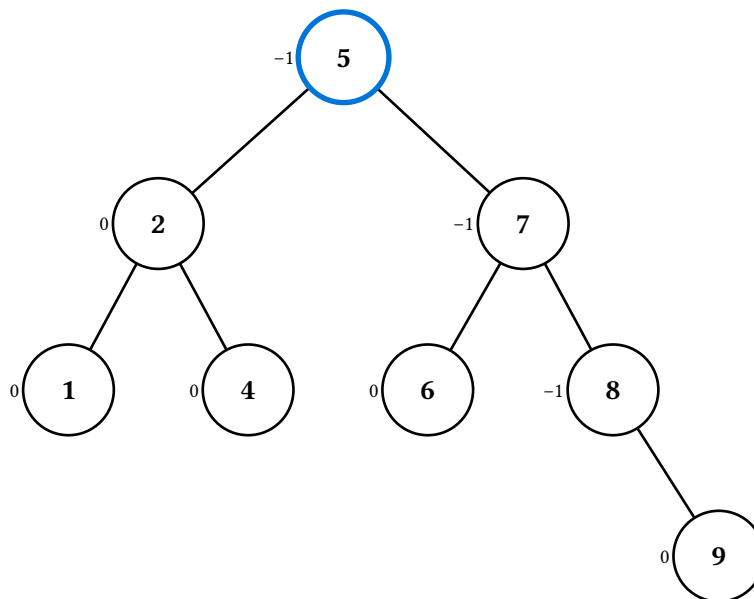
Transfer 2



Remove node



Recompute height = 2



Recompute height = 4

6.4. Rotate

`(t.rotate-display)(c)` is the same six-frame structural rotation animation BST exposes, with heights refreshed in the after-tree. AVL rebalancing in `insert` / `delete` does **not** go through this method — the case-dispatch rotations are applied directly so the animation can narrate the recompute-then-rotate logic.

6.5. Fix-up

`(t.fixup-display)(violation-path)` runs the AVL fix-up climb from a hand-constructed (possibly invalid) tree. The tree is **not** validated: this is intended for teaching configurations that can't arise from a single insert or delete — for example, multiple imbalances on one spine where the inner rotation propagates the imbalance up. The climb walks every proper prefix of `violation-path` deepest-first, emitting the same recompute / check / rotate-zigzag / rotate-finish events that `insert` and `delete` use.

6.6. Traversals

(`t.in-order-display()`), (`t.pre-order-display()`), (`t.post-order-display()`), and (`t.level-order-display()`) work exactly like their BST counterparts — one frame per visit, gradient fill from the op-theme’s traversal-palette, running output sequence in the caption. Pass factors: `true` on any traversal to layer the balance-factor tags under the per-visit highlights.

7. Theming

Starling exposes three theme layers so document authors can match starling’s palette to their own document style:

- **Render theme** (`default-render-theme`) — the structural defaults the renderer falls back on for the unstyled tree: `node-fill`, `node-stroke`, `node-text-fill`, `edge-stroke`, `note-fill`.
- **Op theme** (`default-op-theme`) — the operation-semantic colors and strokes that any data structure’s `*-display` methods use to communicate operation state: `search-stroke`, `attention-stroke`, `success-stroke`, `settled-stroke`, `success-fill`, `danger-stroke`, `reset-stroke`, `traversal-palette`. Shared across data structures — BST search, RBT search, and a future heap delete all read the same `search-stroke`. Strokes are full stroke dicts (`(paint:, thickness:, dash:)`) so any aspect — color, width, dash — is overridable without adding more keys.
- **Per-DS theme** — styling intrinsic to one data structure. RBT has `default-rbt-theme` with `red-fill`, `red-stroke`, `red-text-fill`, `black-fill`, `black-stroke`, `black-text-fill` for the red/black palette. BST has no per-DS theme of its own because BST nodes carry no intrinsic styling.

7.1. Setting a theme for the whole document

Call any combination of `set-render-theme`, `set-op-theme`, and the per-DS setters (e.g. `set-rbt-theme`) once near the top of your document. Each override is state-based and scoped by Typst’s normal layout flow, so it propagates to every subsequent `*-display` call.

```
#import "@preview/starling:0.1.0": BST, set-op-theme, set-render-theme

#set-op-theme((
  search-stroke: (paint: teal, thickness: 2.5pt),
  success-stroke: (paint: olive, thickness: 2.5pt),
  settled-stroke: (paint: olive, thickness: 3.5pt),
  success-fill: olive.lighten(75%),
))
#set-render-theme((node-fill: yellow.lighten(85%)))
```

Pass a partial dictionary — only the keys you list are changed. Unknown keys panic so typos surface immediately rather than silently falling back to defaults.

7.2. Per-call overrides

For one-off variations, every `*-display` method accepts `theme:` and `render-theme:` arguments. A partial dict is merged into the defaults and used in place of the state value for that call only.

```
(t.search-display)(6, theme: (search-stroke: (paint: olive, thickness: 2pt)))
```

7.3. Performance: state-based theming costs a layout pass

Typst’s state machinery is what makes `set-op-theme` / `set-render-theme` work: a `state.update` later in the document can affect renders earlier in document order, so Typst evaluates the document, propagates state, and re-evaluates anything that observed it. In practice that means **using either state setter roughly doubles the compile time of the state-observed portion of the document** — a small fixed cost on top of each tree, plus a full extra layout pass through everything that placed a starling render helper or rendered a frame’s render builder against that state.

Concretely, a 50-tree synthetic benchmark goes from 1.85s to 3.1s when `set-op-theme` is added. The cost scales with document size, not with the number of state reads (one read or a thousand is the same), because the price is “Typst does a second layout pass”, not “the read is slow”.

This is a structural property of Typst’s state model, not something starling can work around without giving up state. If compile speed matters more than the ergonomic win, hoist your theme dict into a plain `let` and pass it to each `*-display` call via the `per-call theme: argument` — that path skips state entirely and runs at the no-state baseline:

```
#let palette = (
  search-stroke: (paint: teal, thickness: 2.5pt),
  success-stroke: (paint: olive, thickness: 2.5pt),
  // ...
)

#starling.stacked((t.search-display)(6, theme: palette))
#starling.stacked((t.insert-display)(5, theme: palette))
// ...
```

This is the usual perf/ergonomics tradeoff: `set-op-theme` is one declaration at the top of the document and “just works”; `per-call theme:` is more typing but avoids the extra pass.

8. Touying composition examples

Starling is layout-agnostic. In a touying deck, the simplest use splats figures into alternatives:

```
#import "@preview/touying:0.7.3": *
#import "@preview/starling:0.2.0" as starling
#import starling: BST

== Searching
#alternatives(..starling.figures((t.search-display)(6)))
```

Each frame becomes a subslide; touying handles the rest.

8.1. Captions in a sidebar

When the visual animation should sit alongside synchronised text elsewhere on the slide, pull the captions out separately:

```
== Searching for 6
#let frames = (t.search-display)(6)
```

```
#grid(columns: 2, column-gutter: 2em,
  alternatives(..starling.figures(frames, caption: false)),
  alternatives(..frames.map(f => f.caption)),
)
```

Both alternatives calls produce the same number of subslides, so they step in lockstep.

8.2. Driving layout from step metadata

`frame.step` is free-form per-method metadata. For example, a slide that wants to colour-code its narration by step kind:

```
#let kind-colors = (compare: blue, inserted: green)

#let captioned-frames = frames.map(f => figure(context {
  let op = starling._op-theme-state.get()
  let rt = starling._render-theme-state.get()
  let color = kind-colors.at(f.step.kind, default: black)
  stack(dir: ttb, spacing: 0.5em,
    (f.render)(op, rt), text(fill: color, f.caption))
}))

#alternatives(..captioned-frames)
```

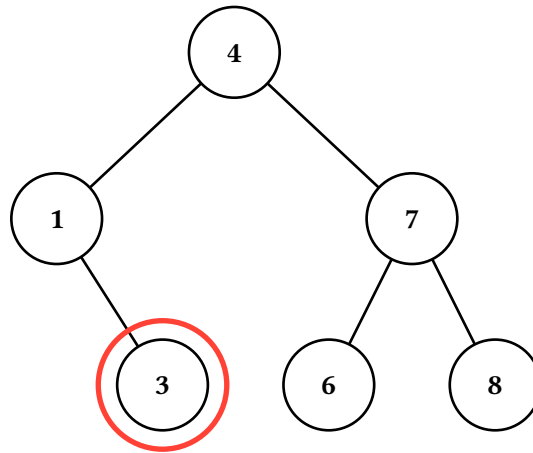
`frame.render` is a builder `(op, rt) => content` rather than pre-baked content, so themes resolve at layout time. The context block above reads the active themes and feeds them in; if you don't need state-driven theming, you can call `(f.render)(starling.default-op-theme, starling.default-render-theme)` without the wrapper.

9. Composing with cetz annotations

For callouts or overlays that need to track specific nodes, drop down to the cetz layer. `draw-tree(tree, snapshot, ...)` emits the tree's cetz drawables without wrapping them in `cetz.canvas`, so you can compose them with your own draw commands in a shared canvas. `path-anchor(path)` translates an L/R path string to the cetz anchor name of the corresponding node.

```
#import "@preview/cetz:0.5.2"

#cetz.canvas({
  starling.draw-tree(t, starling.blank-snapshot())
  import cetz.draw: *
  circle(starling.path-anchor("LR"), radius: 0.85, stroke: red + 2pt)
})
```

The same trick combines with frame canvases: each frame's canvas is itself a `cetz.canvas` block, so for static “annotate the final frame” use you can either render once via `draw-tree` (as above) or extract the `cetz` block from a frame and add to it externally.

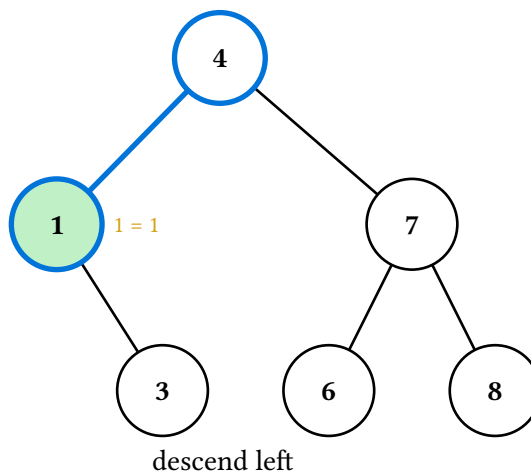
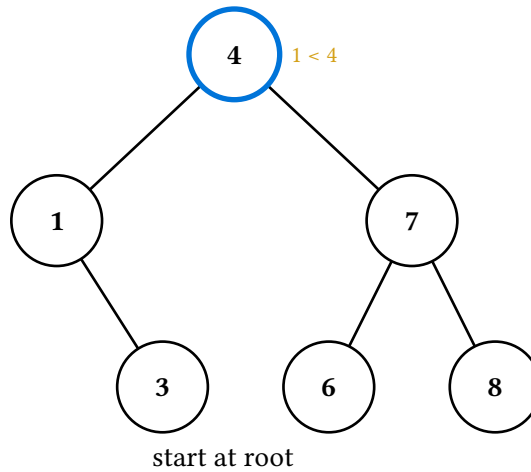
10. The Op command stream

For animations that don't fit the BST's built-in methods — say, a custom hash-table probe sequence or a graph traversal — drop down to the `Op` command stream. Build a sequence of declarative ops and fold them into a renderer with `apply-ops`:

```

#let r = starling.make-renderer(t, sticky: true)
#let r = (r.with-caption)([start at root])
#let r = starling.apply-ops(r, (
  (starling.Op.Highlight.new)(path: "", color: blue),
  (starling.Op.Annotate.new)(path: "", text: [1 < 4]),
  (starling.Op.Commit.new)(
    alt: "Comparing the target 1 against the root 4.",
  ),
),
)
#let r = (r.with-caption)([descend left])
#let r = starling.apply-ops(r, (
  (starling.Op.ClearNotes.new)(),
  (starling.Op.Highlight.new)(path: "L", color: blue),
  (starling.Op.StyleEdge.new)(path: "L", style: (stroke: blue + 2pt)),
  (starling.Op.StyleNode.new)(path: "L", style: (fill: green.lighten(70%))),
  (starling.Op.Annotate.new)(path: "L", text: [1 = 1]),
  (starling.Op.Alt.new)(
    text: "Descended into the left subtree; 1 matches the left child.",
  ),
),
)
#starling.stacked((r.render)())

```



`Op.Commit` closes the current frame, attaches the supplied `alt` text to it, and opens a fresh blank frame. The `alt` argument is required so every frame the command stream produces carries accessible text. The trailing in-progress frame is kept implicitly — no final `Op.Commit` is needed — but use `Op.Alt(text)` (or `r.with-alt(...)` on the returned renderer) to give that last frame its `alt` text too.

`Op.Caption` does *not* exist; captions and step metadata are renderer-level, set via `r.with-caption(...)` / `r.with-step(...)` directly between `Op` batches. The `split` keeps each `Op`'s responsibility narrow (modifying the current snapshot) and matches how the `*-display` methods are written internally.

10.1. Custom shapes and edge anchors

Two `node-style` and `edge-style` keys let you swap the default circular node for a triangle or rectangle and steer where the incoming or outgoing edge connects:

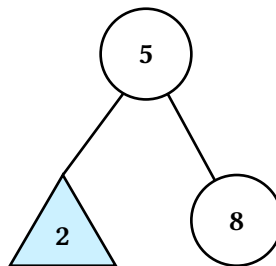
- `shape` (node style) accepts "circle" (default), "triangle" (apex up), or "rectangle". The triangle and rectangle share a 1.4×1.2 bounding box; the circle keeps its 1.2×1.2 footprint.
- `parent-anchor` / `child-anchor` (edge style) accept a cetz anchor name like "north" or "south" and override the edge endpoint on the parent or child side respectively. The default for both stays the empirical fractional-distance trick that looks clean between two circles.

The canonical use case is using a triangle to stand in for an entire subtree — a common pattern when only the top-level shape matters and the details would clutter the slide. Because the triangle's apex sits at

the north anchor of its bounding box, an incoming edge with child-anchor: "north" lands cleanly on the tip:

```
#let t = (starling.BST.new)(value: 5, label: auto, left: none, right: none)
#let t = (t.insert-many)(2, 8, 1, 3)

#let ops = (
  (starling.Op.StyleNode.new)(path: "L", style: (shape: "triangle", fill:
aqua.lighten(60%))),
  (starling.Op.StyleEdge.new)(path: "L", style: (child-anchor: "north")),
  (starling.Op.StyleNode.new)(path: "LL", style: (hide: true)),
  (starling.Op.StyleNode.new)(path: "LR", style: (hide: true)),
  (starling.Op.StyleEdge.new)(path: "LL", style: (hide: true)),
  (starling.Op.StyleEdge.new)(path: "LR", style: (hide: true)),
  (starling.Op.Alt.new)(text: "Left subtree summarised as a triangle."),
)
#let r = starling.apply-ops(starling.make-renderer(t, sticky: true), ops)
#starling.last((r.render)())
```



Shape and anchor are deliberately independent — the library doesn't auto-set child-anchor: "north" when you switch a node to a triangle, because there are legitimate uses for the default endpoint behavior even on non-circular shapes (e.g. a labelled rectangle whose edges you want pulled toward its center rather than flush against its top). When you want flush meeting points, set the anchors yourself.

11. API reference

The remainder of this manual is an auto-generated reference, produced by the tidy package from doc-comments in the source.

11.1. Render helpers

- canvases-only()
- figures()
- last()
- stacked()

11.1.1. canvases-only

Drop captions and step metadata, returning just the array of canvases. Useful when you want to lay out captions yourself — e.g. alongside other slide content in a custom touying layout.

Like `figures()`, this returns one piece of content per frame and therefore can't amortize theme state reads — each canvas resolves state independently. If you're calling this on hot paths under a state-set theme, prefer per-call theme: arguments on the `*-display` method.

11.1.1.1. Parameters

```
canvases-only(frames: array) -> array
```

11.1.1.1.1. frames array

The frame array returned by any `*-display` method.

11.1.2. figures

Build an array of figure content, one per frame, suitable for splatting into `touying's alternatives(...)` so each frame becomes its own subslide. `Starling` itself does not depend on `touying` — the result is just an array of standard Typst figures.

Unlike `last()` / `stacked()`, this helper can't collapse theme state reads to one — each figure is an independent piece of content laid out separately (`touying` splits them into subslides), so each figure's body resolves theme state in its own context block. In many-tree documents that rely on `set-op-theme` / `set-render-theme`, prefer per-call theme: arguments on the `*-display` method to skip state altogether when you're using figures.

11.1.2.1. Parameters

```
figures(  
  frames: array,  
  caption: bool,  
  alt: auto str,  
  caption-spacing: length  
) -> array
```

11.1.2.1.1. frames array

The frame array returned by any `*-display` method.

11.1.2.1.2. caption bool

Whether to stack each frame's caption below its canvas inside the figure body.

Default: `true`

11.1.2.1.3. alt auto or str

Alt-text override applied to every frame. auto uses each frame's alt field, falling back to the caption (if it's a plain string) and then to a generic "Animation frame N" placeholder.

Default: auto

11.1.2.1.4. caption-spacing length

Spacing between canvas and caption when caption: true.

Default: 0.5em

11.1.3. last

Final frame as a single piece of content. Useful for static renderings in print or for “just show me the end state” usage.

The inline node annotations (drawn next to nodes in the cetz canvas) already label what the final step did, so the textual caption is suppressed by default. Pass caption: true to stack the caption below the canvas.

The returned content is wrapped in an alt-tagged figure for accessibility. The alt text comes from the frame's alt field; pass an explicit string to alt to override.

11.1.3.1. Parameters

```
last(  
  frames: array,  
  caption: bool,  
  spacing: length,  
  alt: auto str  
) -> content
```

11.1.3.1.1. frames array

The frame array returned by any *-display method.

11.1.3.1.2. caption bool

Whether to render the step's caption below the canvas.

Default: false

11.1.3.1.3. spacing `length`

Vertical spacing between canvas and caption when `caption: true`.

Default: `0.5em`

11.1.3.1.4. alt `auto` or `str`

Alt-text override. `auto` uses the frame's `alt` field.

Default: `auto`

11.1.4. stacked

All frames stacked vertically as one block of content. Useful for handouts that want to show the full animation in a single figure. Captions are on by default since the stack is read as a sequence — the caption text annotates which step each tree corresponds to.

Each frame is wrapped individually in an alt-tagged figure so screen-reader users hear the per-step narration. Pass a string to `alt` to set the same override on every frame.

11.1.4.1. Parameters

```
stacked(  
  frames: array,  
  caption: bool,  
  spacing: length,  
  caption-spacing: length,  
  alt: auto str  
) -> content
```

11.1.4.1.1. frames `array`

The frame array returned by any `*-display` method.

11.1.4.1.2. caption `bool`

Whether to include each frame's caption below its canvas.

Default: `true`

11.1.4.1.3. spacing `length`

Vertical spacing between frames.

Default: `1em`

11.1.4.1.4. caption-spacing `length`

Spacing between each canvas and its caption.

Default: `0.5em`

11.1.4.1.5. alt `auto` or `str`

Alt-text override applied to every frame. `auto` uses each frame's `alt` field.

Default: `auto`

11.2. Animation kernel

- `apply-ops()`
- `blank-snapshot()`
- `concat-frames()`
- `draw-tree()`
- `make-renderer()`
- `path-anchor()`
- `set-render-theme()`

Variables

- `PathId`
- `NodeStyle`
- `EdgeStyle`
- `default-render-theme`
- `RenderTheme`
- `Snapshot`
- `Frame`
- `TreeRenderer`
- `Op`

11.2.1. apply-ops

Fold a sequence of `Op` values into a renderer, returning the updated renderer. The trailing in-progress frame is kept — no explicit `Op.Commit` is needed after the last batch of edits, but attach alt text to that trailing frame via `Op.Alt` (or `r.with-alt(...)` on the returned renderer) so it has accessible text like the committed frames do.

11.2.1.1. Parameters

```
apply-ops(  
  renderer: TreeRenderer,  
  ops: array  
) -> TreeRenderer
```

11.2.1.1.1. renderer `TreeRenderer`

The renderer to apply ops to.

11.2.1.1.2. ops array

Sequence of Op values to fold left-to-right.

11.2.2. blank-snapshot

Build a fresh Snapshot with no node or edge style overrides. Useful when you want to render a tree once with draw-tree and no per-frame state.

11.2.2.1. Parameters

`blank-snapshot()` -> Snapshot

11.2.3. concat-frames

Stitch frame arrays from multiple renderers (or pre-rendered arrays) into one flat Array(Frame). Use this when an animation spans a tree-shape change (rotation, deletion, insertion) and one renderer can't hold all the frames — one renderer per shape, joined at the boundary.

Accepts a mix of TreeRenderer instances and already-rendered arrays; the former are render-ed in place.

11.2.3.1. Parameters

`concat-frames(..parts)` -> array

11.2.3.1.1. ..parts

Variadic mix of renderers and frame arrays.

11.2.4. draw-tree

Emit the cetz draw commands for one styled snapshot of tree, *without* wrapping them in a `cetz.canvas`. Caller is responsible for the canvas wrap. Use this when you want to add your own cetz annotations alongside the tree — for example, callouts anchored at specific nodes.

The whole tree is wrapped in a named cetz group (name, default "tree"); each non-phantom node has an inner-name of <node-prefix><cetz-tree-name> where the cetz-tree name is "0" for the root, "0-0" for L, "0-1" for R, "0-0-1" for LR, and so on. Fully qualified anchor names therefore look like "tree.node-0-0-1" — use `path-anchor()` to compute them from an L/R path string.

11.2.4.1. Parameters

```
draw-tree(  
  tree: dictionary,  
  snapshot: Snapshot,  
  default-node-style: dictionary,  
  default-edge-style: dictionary,  
  render-theme: dictionary,  
  name: str,  
  node-prefix: str,  
  grow: float,  
  spread: float  
) -> content
```

11.2.4.1.1. tree dictionary

The tree to render — any value with value, label, left, and right fields (e.g. a BST instance). label of auto falls back to `str(value)` any other value (string, image, content) is rendered as the node's drawn label.

11.2.4.1.2. snapshot Snapshot

Style overlay for this snapshot. Use `blank-snapshot()` for an unstyled tree.

11.2.4.1.3. default-node-style dictionary

Default node-style overrides applied before per-node overrides.

Default: `(:)`

11.2.4.1.4. default-edge-style dictionary

Default edge-style overrides applied before per-edge overrides.

Default: `(:)`

11.2.4.1.5. render-theme dictionary

Structural defaults — the lowest layer of the style chain. Defaults to `default-render-theme` pass an already-merged dict (e.g. read from `set-render-theme`'s state inside a context block) to apply user overrides.

Default: `default-render-theme`

11.2.4.1.6. **name** `str`

Outer group name for the whole tree. Must match the `tree-name` argument to `path-anchor()` for anchors to resolve.

Default: `"tree"`

11.2.4.1.7. **node-prefix** `str`

Prefix attached to each node's `cetz-tree` internal name. Must match the `prefix` argument to `path-anchor()`.

Default: `"node- "`

11.2.4.1.8. **grow** `float`

Depth-direction layout factor passed to `cetz-tree`. Values below 1 shorten edges between levels; values above 1 stretch them. Node sizes are unaffected.

Default: `1`

11.2.4.1.9. **spread** `float`

Sibling-direction layout factor passed to `cetz-tree`. Values below 1 pull siblings together; values above 1 push them apart. Node sizes are unaffected.

Default: `1`

11.2.5. **make-renderer**

Build a `TreeRenderer` seeded with one blank initial frame.

With `sticky: true` (the default), each new frame pushed via `r.push-frame()` starts from the previous frame's style snapshot — so highlights accumulate over the course of an animation. Set `sticky: false` to start each frame from a clean slate.

With `theme: auto` (the default), each rendered canvas reads the active render theme from state at layout time, so a `set-render-theme(...)` earlier in the document propagates. Pass an explicit dictionary to bake a theme in and skip the state lookup.

11.2.5.1. Parameters

```
make-renderer(  
  tree: dictionary,  
  default-node-style: dictionary,  
  default-edge-style: dictionary,  
  sticky: bool,  
  theme: auto dictionary  
) -> TreeRenderer
```

11.2.5.1.1. tree dictionary

The tree to render — any value with value, label, left, and right fields. See `draw-tree()` for how label is rendered.

11.2.5.1.2. default-node-style dictionary

Default node-style overrides applied before per-snapshot overrides.

Default: `(:)`

11.2.5.1.3. default-edge-style dictionary

Default edge-style overrides applied before per-snapshot overrides.

Default: `(:)`

11.2.5.1.4. sticky bool

When `true`, new frames inherit the previous frame's style. When `false`, each new frame starts blank.

Default: `true`

11.2.5.1.5. theme auto or dictionary

Render-theme override for this renderer. `auto` reads the active render-theme from state at layout time. A dict is merged into `default-render-theme` once and baked in.

Default: `auto`

11.2.6. path-anchor

Translates a starling path-id (L/R string rooted at `"`) to the fully-qualified cetz anchor name produced by `draw-tree()`. Path `"` maps to the root anchor; `"L"/"R"` map to first-level children; and so on.

The `tree-name` and `prefix` arguments must match the name and node-prefix passed to `draw-tree`.

11.2.6.1. Parameters

```
path-anchor(  
  path: str,  
  tree-name: str,  
  prefix: str  
) -> str
```

11.2.6.1.1. path str

L/R path string identifying a node (" " = root, "LR" = root's left child's right child, etc.).

11.2.6.1.2. tree-name str

Outer-group name; must match draw-tree's name.

Default: "tree"

11.2.6.1.3. prefix str

Per-node prefix; must match draw-tree's node-prefix.

Default: "node- "

11.2.7. set-render-theme

Override one or more render-theme keys for the rest of the document (state-based, scoped by Typst's normal layout flow). Pass a partial dictionary — only the keys you list are changed; the rest stay at their current values. Unknown keys panic.

11.2.7.1. Parameters

```
set-render-theme(theme)
```

11.2.8. PathId

Typsy refinement: a string built only from "L" and "R" characters. The empty string is the root; "L" is the root's left child; "LR" is the root's left child's right child; and so on.

11.2.9. NodeStyle

Typsy refinement: a dictionary of node-style overrides. Recognised keys are fill, stroke, text-fill, note, note-fill, hide, shape, tag, label, and materialize. shape accepts the string "circle" (default), "triangle" (apex up; useful as a subtree-summary marker), or "rectangle". The triangle and rectangle

share a 1.4×1.2 bounding box; the circle keeps its 1.2×1.2 footprint. Named cetz anchors on the node's group (north, south, east, west) follow each shape's bounding box. `tag` is a small piece of content drawn just outside the west of the node — useful for compact per-node annotations that don't compete with the label or the operation note slot (e.g. the RBT black-height bits enabled by `display(bits: true)`). `label` replaces the node's rendered label (otherwise derived from the tree node's `label` field, or `str(value)` when that's auto). `materialize: true` draws an otherwise-phantom slot as a real node — useful for one-off pedagogical animations that need to expose a nil child (e.g. rendering $\emptyset$ at a just-deleted leaf's position). Phantoms only exist at paths the tree either naturally generates as layout-balance siblings or where an edge style sets `force-show: true` materializing without one of those has no node to attach to.

11.2.10. EdgeStyle

Typsy refinement: a dictionary of edge-style overrides. Recognised keys are `stroke`, `note`, `note-fill`, `mark`, `hide`, `parent-anchor`, `child-anchor`, and `force-show`. The two `*-anchor` keys accept a cetz anchor name (e.g. "north") and override the default fractional-distance endpoint on the parent or child side respectively; useful for connecting edges to non-circular shapes (e.g. `child-anchor:`

"north" to land on a triangle's apex). `force-show: true` renders an edge even when one endpoint is a phantom — used by the RBT delete animation to draw a stub edge into a now-nil position when marking double-black.

11.2.11. default-render-theme

Default render theme. The structural defaults the renderer falls back on when neither a snapshot nor `default-node-style/ default-edge-style` specifies otherwise.

11.2.12. RenderTheme

Typsy refinement: a dictionary whose keys are a subset of the render-theme keys (`node-fill`, `node-stroke`, `node-text-fill`, `edge-stroke`, `note-fill`).

11.2.13. Snapshot

Typsy class representing one sparse style overlay — the per-node and per-edge style overrides for a single frame. Build one via `blank-snapshot()` and chain its `style-node / style-edge / note-node / note-edge / clear-notes` methods. Snapshots are rarely constructed directly — they're managed by `make-renderer()` and the BST's `*-display` methods.

11.2.14. Frame

Typsy class representing one frame in an animation. Fields and methods:

- `render(op-theme, render-theme)` — method that, given resolved theme dicts, produces the cetz canvas for this frame. Theming-aware deferral lives here: by carrying a builder rather than pre-baked content, the `lib.typ` helpers (`last`, `stacked`, `figures`) can resolve theme state once per call instead of once per frame, which is meaningfully faster in many-tree documents. Per-data-structure themes (e.g. the RBT palette) are read inside each frame’s builder rather than passed in — the helpers only carry the two universal layers.
- `caption` — optional textual track for the step (possibly none).
- `step` — optional free-form per-method metadata (possibly none).
- `alt` — optional accessible text describing the frame (possibly none). Carried verbatim into the `alt:` argument on the Typst figure wrapping the canvas. The first frame of each operation includes the full tree structure (via the `BST describe` method) so screen-reader users have a baseline; subsequent frames describe only the per-step change.

Direct rendering pattern for custom layouts:

```
context {  
  let op = _op-theme-state.get()  
  let rt = _render-theme-state.get()  
  ... (frame.render)(op, rt) ...  
}
```

Or simply pass the frame array to one of the `lib.typ` helpers, which handle the state wrap for you.

The internal `_builder` field carries the actual rendering closure wrapped in a singleton dict so typsy’s auto-self-injection doesn’t fire on it; treat it as private and call `render` instead.

11.2.15. TreeRenderer

Typsy class accumulating an animation. Holds the tree, parallel arrays of snapshots / captions / steps / alts (one entry per frame), and default node/edge styles. Methods include `push-frame()`, `patch(fn)`, `push-with-node(path, ..style)`, `push-with-edge(path, ..style)`, `push-note-node(path, txt)`, `push-note-edge(path, txt)`, `with-caption(c)`, `with-step(s)`, `with-alt(a)`, and `render()`. Build one via `make-renderer()`.

11.2.16. Op

Typsy enumeration describing a declarative command stream for building animations. Variants:

- `Op.Highlight(path, color)` — set a node’s stroke to `color + 2pt`.
- `Op.Annotate(path, text)` — attach an inline note to a node (drawn in-canvas).
- `Op.StyleNode(path, style)` — arbitrary node-style overrides.

- `Op.StyleEdge(path, style)` — arbitrary edge-style overrides.
- `Op.Commit(alt)` — finalise the current frame, attach `alt` text to it, and open a fresh blank frame. `alt` is required so every frame the command stream produces carries accessible text.
- `Op.Alt(text)` — set `alt` text on the in-progress frame without committing. Primary use is the trailing frame, which has no following `Op.Commit` to carry its `alt`; can also be used to set `alt` earlier in a frame's lifecycle if convenient.
- `Op.ClearNotes()` — drop all notes from the current frame's snapshot.

Captions and step metadata are renderer-level rather than snapshot-level: set them between `Op` batches via `r.with-caption(...)` / `r.with-step(...)`, not through an `Op`. Fold a sequence of `Ops` into a renderer with `apply-ops()`.